Proyecto CyberGuard: Plataforma de análisis y detección de comportamientos anómalos en redes

Segunda entrega: diseño, desarrollo y avance del sistema

Sergio Esteban Veloza Gonzalez, Luifer Andres Hernandez Lambraño,

Marcos Antonio Pacheco Bautista y Rafael Antonio Rodríguez Gamarra

Programa de Ingeniería de Sistemas, Facultad de Ingeniería

Corporación Unificada Nacional de Educación Superior

Programación avanzada

Docente: Faber Alemán

Sincelejo, Colombia

5 de noviembre de 2026

Contenido

1. Introducción 5
2. Descripción del problema 6
2.1 Formulación del problema 7
2.2 Delimitación 7
3. Justificación 7
4. Objetivo general y objetivos específicos 9
4.1 Objetivo general 9
4.2 Objetivos específicos 9
5. Alcance 10
5.1 Alcance 10
5.2 Limitaciones 10
6. Requerimientos funcionales y no funcionales 11
6.1 Requerimientos funcionales 11
6.2 Requerimientos no funcionales 13
7. Usuarios o actores del sistema 14
8. Diagramas correspondientes al proyecto 17
8.1 Diagrama de casos de uso 17
8.2 Diagrama de clases del dominio 18
8.3 Diagrama de clases del motor de análisis 19
8.4 Modelo entidad-relación 21
8.5 Diagrama de secuencia: registrar un evento 22
8.6 Diagrama de actividades del análisis 23
8.7 Diagrama de estados de la alerta 25
8.8 Diagrama de despliegue 25
9. Diseño de la solución 26
9.1 Flujo general 26
9.2 Modelo de datos 26
9.3 Diseño del motor de reglas 27
9.4 Clasificación de riesgo 28
9.5 Decisiones de diseño 30
9.6 Diseño de la interfaz 30
10. Tecnologías y herramientas utilizadas 31
11. Arquitectura propuesta 33

12. Descripción de las funcionalidades desarrolladas 37
12.1 Autenticación y control de acceso 37
12.2 Gestión de usuarios (administrador) 37
12.3 Inventario de dispositivos 38
12.4 Registro y consulta de eventos 38
12.5 Motor de análisis y clasificación 38
12.6 Alertas y seguimiento 39
12.7 Gestión de reglas 39
12.8 Panel de información 39
12.9 Reportes 39
12.10 Página informativa 40
12.11 Datos de demostración 40
13. Avance del desarrollo 41
13.1 Avance por fases de la planificación 41
13.2 Avance por módulo 41
13.3 Historial del repositorio 42
14. Evidencias del sistema 42
14.1 Acceso 43
14.2 Panel 47
14.3 Dispositivos 48
14.4 Eventos 52
14.5 Alertas 54
14.6 Reglas 57
14.7 Reportes 59
14.8 Usuarios 60
14.9 Errores 63
15. Pruebas realizadas hasta el momento 64
15.1 Pruebas automáticas del backend 64
15.1.1 Pruebas automáticas del cliente 65
15.2 Pruebas funcionales manuales 67
15.3 Validaciones comprobadas 68
16. Problemas encontrados y cómo fueron solucionados 68
17. Funcionalidades pendientes para la tercera entrega 71
18. Conclusiones parciales 72

19. Referencias 74
Anexo A. Índice de figuras 77

1. Introducción

El crecimiento de las redes informáticas ha permitido que empresas, instituciones educativas y otras organizaciones conecten una gran cantidad de dispositivos para compartir información y utilizar servicios de manera rápida. Este crecimiento también trae retos para identificar, supervisar y gestionar los eventos que pueden afectar la seguridad de la información.

En una infraestructura de red pueden coexistir computadores, teléfonos, servidores, impresoras, cámaras y otros dispositivos que generan distintos tipos de actividad. Sin mecanismos para registrar y analizar esos eventos, resulta difícil saber cuándo una actividad se sale de los patrones esperados: un dispositivo desconocido, varios intentos de conexión seguidos, actividad repetitiva o accesos a muchos puertos en poco tiempo.

La gestión de eventos de seguridad requiere identificarlos, analizarlos y decidir qué hacer ante las situaciones que puedan representar un riesgo (National Institute of Standards and Technology [NIST], 2012). Para que ese análisis sea posible, los eventos deben registrarse de forma ordenada, con su marca de tiempo y su origen, y revisarse con periodicidad; Kent y Souppaya (2006) señalan que una gestión de registros deficiente impide detectar a tiempo los incidentes y reconstruir lo ocurrido. Los marcos actuales de ciberseguridad también resaltan la importancia de identificar y analizar los riesgos asociados con los sistemas y activos tecnológicos (NIST, 2024).

A partir de esta problemática surge CyberGuard, una aplicación web formada por una API REST en Java con Spring Boot y un cliente web en React, para registrar, analizar y clasificar eventos de una red autorizada. En la primera entrega se definieron el problema, los objetivos, el alcance, los requerimientos y la arquitectura. Esta segunda entrega documenta el diseño detallado, la solución construida, el estado del desarrollo, las pruebas ejecutadas, los

problemas resueltos y lo que queda para la tercera entrega. Todo lo descrito corresponde al código fuente actual del repositorio. Durante esta entrega la aplicación pasó de un monolito con vistas renderizadas en el servidor a dos aplicaciones separadas (API y cliente); el documento describe la versión actual.

2. Descripción del problema

Una organización puede tener muchos dispositivos conectados a la misma infraestructura de red. A medida que aumentan los equipos y servicios, también aumenta la cantidad de eventos que se producen cada día.

El problema no es la existencia de muchos dispositivos, sino la dificultad para decidir cuándo una actividad merece atención. Revisar a mano grandes cantidades de eventos toma tiempo y retrasa la identificación de ciertos comportamientos.

Por ejemplo, un administrador puede encontrar un dispositivo que aparece por primera vez en la red, una dirección IP con una cantidad inusual de solicitudes o un equipo que intenta conectarse a varios puertos en pocos minutos. Estas situaciones no prueban que exista un ataque, pero sí requieren revisión. La identificación y el análisis de eventos son una parte central de la gestión de incidentes, porque permiten reunir información y decidir si hace falta una respuesta (NIST, 2012). Los sistemas de detección de intrusos basados en anomalías parten de la misma idea: comparar la actividad observada con un comportamiento esperado y señalar las desviaciones (Scarfone y Mell, 2007; Chandola et al., 2009). En la revisión de Liao et al. (2013), los métodos de detección se agrupan en basados en firmas, en anomalías y en análisis de protocolos con estado; los basados en anomalías detectan comportamientos nuevos, pero dependen de definir bien qué es «normal» y tienden a producir falsos positivos. CyberGuard

adopta una variante sencilla y explicable de ese enfoque: umbrales explícitos sobre la actividad de cada dirección IP dentro de una ventana de tiempo.

Cuando la información se revisa solo de forma manual, el administrador tiene dificultades para relacionar eventos, dispositivos, fechas, frecuencias y condiciones. Eso produce retrasos al identificar comportamientos potencialmente anómalos.

2.1 Formulación del problema

¿Cómo desarrollar una plataforma basada principalmente en Java que permita registrar, analizar y clasificar eventos de una red autorizada, para facilitar la identificación de comportamientos potencialmente anómalos y generar alertas para su seguimiento?

2.2 Delimitación

CyberGuard no pretende sustituir plataformas profesionales de seguridad (IDS, SIEM o EDR) ni funcionar como un sistema empresarial de detección y respuesta. Es una solución académica que aplica programación avanzada, estructuras de datos, algoritmos, bases de datos, arquitectura de software y fundamentos de ciberseguridad a un problema concreto.

3. Justificación

CyberGuard se propone como una alternativa académica y tecnológica para abordar el monitoreo y análisis de eventos de seguridad en redes.

Desde la Ingeniería de Sistemas, el proyecto integra en una misma solución la programación orientada a objetos, las estructuras de datos, los algoritmos, las bases de datos, la arquitectura de software y la seguridad informática.

Java se eligió como lenguaje principal porque permite construir una estructura orientada a objetos y organizar la aplicación en componentes (Oracle, s. f.). En el código actual esto se ve en

el uso de interfaces, *records*, enumeraciones, inyección de dependencias y el patrón estrategia (Gamma et al., 1994).

El proyecto no se limita a guardar y consultar datos. Incorpora un **motor de análisis basado en reglas**: cada evento registrado se compara con reglas configurables y, si alguna se cumple, el sistema suma una puntuación, clasifica el riesgo y crea una alerta. Por ejemplo, si una dirección IP registra ocho intentos de conexión en cinco minutos, la regla de múltiples intentos se activa y el hecho queda registrado para revisión.

En la práctica, la herramienta sirve de apoyo a un usuario autorizado que necesite consultar rápidamente los dispositivos registrados, los eventos producidos y las alertas que requieren más atención.

Desde el punto de vista académico, el proyecto exige planificar una solución por módulos y una arquitectura que facilite la integración y la evolución del sistema.

El proyecto también aplica principios de seguridad en el propio software: control de acceso por roles, contraseñas cifradas, autenticación sin estado con tokens JWT firmados y validación de entradas, en línea con los riesgos descritos por el OWASP Top 10 (OWASP Foundation, 2021a) y los controles de verificación de OWASP ASVS (OWASP Foundation, 2021b).

Finalmente, el desarrollo se realiza solo sobre redes, dispositivos y datos para los que existe autorización. CyberGuard no escanea redes, no captura tráfico y no bloquea conexiones: los eventos se registran en el sistema y se analizan dentro de él.

4. Objetivo general y objetivos específicos

4.1 Objetivo general

Desarrollar una plataforma de software denominada CyberGuard, basada principalmente en Java, que permita registrar y analizar eventos de una red autorizada mediante reglas de detección, con el propósito de identificar comportamientos potencialmente anómalos, clasificarlos según su nivel de riesgo y generar alertas que faciliten su seguimiento.

4.2 Objetivos específicos

1. Identificar y documentar los requerimientos funcionales y no funcionales de CyberGuard, y diseñar con base en ellos una arquitectura modular y una base de datos que soporten la gestión de usuarios, dispositivos, eventos, reglas y alertas.
2. Definir reglas iniciales de análisis y un mecanismo de clasificación de riesgo para los eventos de una red autorizada, junto con una interfaz de consulta de dispositivos, eventos, alertas e historial.
3. Establecer una metodología de trabajo que distribuya las actividades entre los cuatro integrantes y definir los tipos de pruebas para verificar los componentes del sistema.

Tabla 1

Estado de los objetivos en esta entrega

Objetivo	Estado	Evidencia en el proyecto
1. Requerimientos, arquitectura y base de datos	Cumplido	Secciones 6, 9 y 11; paquetes `domain`, `repository`, `service`, `web`; `Docs/base-de-datos.md`
2. Reglas, clasificación de riesgo e interfaz de consulta	Cumplido	Paquete `analysis`, seis reglas en `analysis/reglas`, `ClasificadorRiesgo`, API REST en `web/` y cliente React con 15 páginas en `frontend/src/pages`
3. Metodología y pruebas	En curso	Scrum por sprints; 62 pruebas del backend y 24 del cliente aprobadas; cobertura con JaCoCo y más pantallas del cliente pendientes (sección 17)

5. Alcance

5.1 Alcance

CyberGuard es una aplicación web académica para registrar, analizar y clasificar eventos de una red autorizada. El alcance implementado hasta esta entrega comprende:

- Autenticación con usuario y contraseña que emite un token JWT, y gestión de usuarios con tres roles.
- API REST en Spring Boot y cliente web en React que la consume.
- Inventario de dispositivos de la red (IP, MAC, nombre, tipo, estado, fecha de registro y última actividad).
- Registro manual de eventos y consulta del historial con filtros por IP, tipo y rango de fechas.
- Motor de análisis con seis reglas de detección configurables.
- Clasificación del riesgo por puntuación acumulada (bajo, medio, alto, crítico).
- Generación automática de alertas y seguimiento por estados con nota del revisor.
- Panel con indicadores y actividad reciente.
- Reportes de eventos y alertas con descarga en CSV.
- Base de datos MySQL/MariaDB creada y actualizada por Hibernate.
- Página informativa interactiva (`/info`) que explica la arquitectura, la base de datos, el motor de reglas y la seguridad.
- Datos de demostración que se cargan solos la primera vez.
- Pruebas automáticas del backend sobre una base H2 en memoria y del cliente con Vitest.

5.2 Limitaciones

- No sustituye plataformas profesionales de seguridad informática.

- No realiza ataques reales ni actividades para vulnerar sistemas.
- No monitorea redes sin autorización, no captura tráfico y no ejecuta escaneos: los eventos se registran en el sistema.
- No bloquea automáticamente dispositivos ni conexiones.
- No incluye inteligencia artificial ni aprendizaje automático; la detección es por reglas.
- No hace análisis de malware.
- Las reglas se limitan a los seis comportamientos implementados.
- El token JWT no se puede revocar antes de que expire (120 minutos): si un administrador desactiva una cuenta o le cambia el rol, la sesión abierta conserva sus permisos hasta que el token caduca.
- El rendimiento depende de los recursos del entorno académico donde se ejecute.
- La versión actual demuestra los conceptos del proyecto y no está pensada para infraestructuras empresariales de gran escala.

6. Requerimientos funcionales y no funcionales

6.1 Requerimientos funcionales

La columna «Implementación» indica la clase o ruta del código que atiende cada requerimiento.

Tabla 2

Requerimientos funcionales

Código	Requerimiento	Descripción	Implementación	Estado
RF01	Autenticación	Los usuarios registrados ingresan con usuario y contraseña; un usuario inactivo no puede entrar.	`SecurityConfig`, `UsuarioDetailsService`, `AuthController` (`POST /api/auth/login`)	Implementado

Código	Requerimiento	Descripción	Implementación	Estado
RF02	Gestión de usuarios	El administrador crea, modifica, consulta y desactiva usuarios. Debe quedar al menos un administrador activo y nadie puede desactivarse a sí mismo.	`UsuarioService`, `UsuarioController`, `/api/usuarios`	Implementado
RF03	Gestión de dispositivos	Registrar, actualizar y consultar dispositivos. La IP es única y la MAC se normaliza en mayúsculas.	`DispositivoService`, `/api/dispositivos`	Implementado
RF04	Registro de eventos	Almacenar eventos con IP, tipo, puerto, fecha, detalle e importancia. El acceso a puerto exige número de puerto y la fecha no puede ser futura.	`EventoService.registrar`, `POST /api/eventos`	Implementado
RF05	Consulta de eventos	Consultar el historial filtrado por IP parcial, tipo y rango de fechas.	`EventoService.buscar`, `GET /api/eventos`	Implementado
RF06	Análisis de eventos	Cada evento registrado se evalúa con las reglas activas.	`MotorAnalisis.analizar`	Implementado
RF07	Generación de alertas	Si al menos una regla se cumple se crea una alerta pendiente ligada al evento y a las reglas cumplidas.	`AlertaService.crear`	Implementado
RF08	Clasificación de riesgo	La suma de puntos (0 a 100) se traduce en bajo, medio, alto o crítico.	`ClasificadorRiesgo`	Implementado
RF09	Gestión de reglas	El administrador ajusta puntuación, umbral, ventana y si la regla está activa; el analista solo consulta.	`ReglaService`, `/api/reglas`	Implementado
RF10	Historial	Se conservan todos los eventos y alertas con su fecha, y se consultan con filtros.	`/api/eventos`, `/api/alertas`	Implementado
RF11	Panel de información	Conteo de dispositivos, eventos, alertas, alertas altas y críticas; últimos 8 eventos y últimas 5 alertas.	`PanelService`, `/api/panel`	Implementado
RF12	Reportes	Consulta y descarga en CSV de eventos y alertas con filtros.	`ReporteService`, `/api/reportes`	Implementado
RF13	Seguimiento de alertas	El analista o el administrador cambian el estado de la alerta y dejan una nota.	`AlertaService.actualizarEstado`, `PUT /api/alertas/{id}/estado`	Implementado (nuevo en esta entrega)

6.2 Requerimientos no funcionales

Tabla 3

Requerimientos no funcionales

Código	Atributo	Descripción	Cómo se atiende en el código
RNF01	Seguridad	Proteger la información con control de acceso.	Spring Security con roles por endpoint, tokens JWT firmados con HS256 (clave de al menos 32 bytes), API sin estado, CORS limitado a los orígenes configurados, BCrypt para contraseñas, validación de entradas con Jakarta Validation y respuestas 401/403 sin datos.
RNF02	Rendimiento	Procesar eventos sin retrasos innecesarios.	El análisis se hace una sola vez al registrar el evento; solo se consulta el historial de la misma IP dentro de la ventana de cada regla, hasta la fecha del evento; las listas no vuelven a analizar.
RNF03	Escalabilidad	Agregar reglas y módulos sin rehacer el sistema.	Patrón estrategia: una regla nueva es una clase que implementa `ReglaDeteccion` más una fila en `reglas`. El motor no cambia.
RNF04	Mantenibilidad	Componentes independientes y fáciles de modificar.	Paquetes por responsabilidad (`web`, `service`, `analysis`, `repository`, `domain`), inyección por constructor, formularios y DTO separados de las entidades; cliente y API independientes.
RNF05	Usabilidad	Interfaz clara y comprensible.	Menú por secciones, filtros en listas, mensajes de validación en español junto a cada campo, botones ocultos según el rol, IP autocompletada al elegir dispositivo.
RNF06	Disponibilidad	Disponible durante los periodos de operación.	La API (Tomcat embebido) y el cliente se inician cada uno con un comando; el cliente muestra mensajes propios cuando la API no responde y ante páginas inexistentes.
RNF07	Integridad	Relaciones consistentes entre las tablas.	Claves foráneas, columnas únicas (`usuario`, `correo`, `direccion_ip`, `codigo`), enums guardados como texto, operaciones en transacciones (`@Transactional`).
RNF08	Portabilidad	Ejecutar en distintos equipos del laboratorio.	Maven Wrapper (`mvnw.cmd`) descarga Maven y dependencias; `npm install` resuelve el cliente; se necesita JDK 21+, Node.js 20+ y MySQL/MariaDB. La conexión se configura en `backend/.env`.

7. Usuarios o actores del sistema

CyberGuard maneja tres roles de usuario (enum `Rol`) y un actor interno. Los permisos son acumulativos: el analista puede todo lo que puede consulta, y el administrador todo lo que puede el analista.

Tabla 4

Usuarios o actores del sistema

Actor	Tipo	Responsabilidades	Usuario de demostración
Administrador	Humano, rol `ADMINISTRADOR`	Todo lo del analista, más crear y editar usuarios y ajustar las reglas de detección.	`admin` / `Admin123*`
Analista	Humano, rol `ANALISTA`	Registrar y editar dispositivos, registrar eventos, cambiar el estado de las alertas, consultar reglas y descargar reportes.	`analista` / `Analista123*`
Consulta	Humano, rol `CONSULTA`	Ver el panel, los dispositivos, los eventos y las alertas. No crea ni modifica datos ni abre reportes.	`consulta` / `Consulta123*`
Motor de análisis	Sistema (interno)	Evalúa cada evento nuevo con las reglas activas, clasifica el riesgo y crea la alerta.	—

Tabla 5

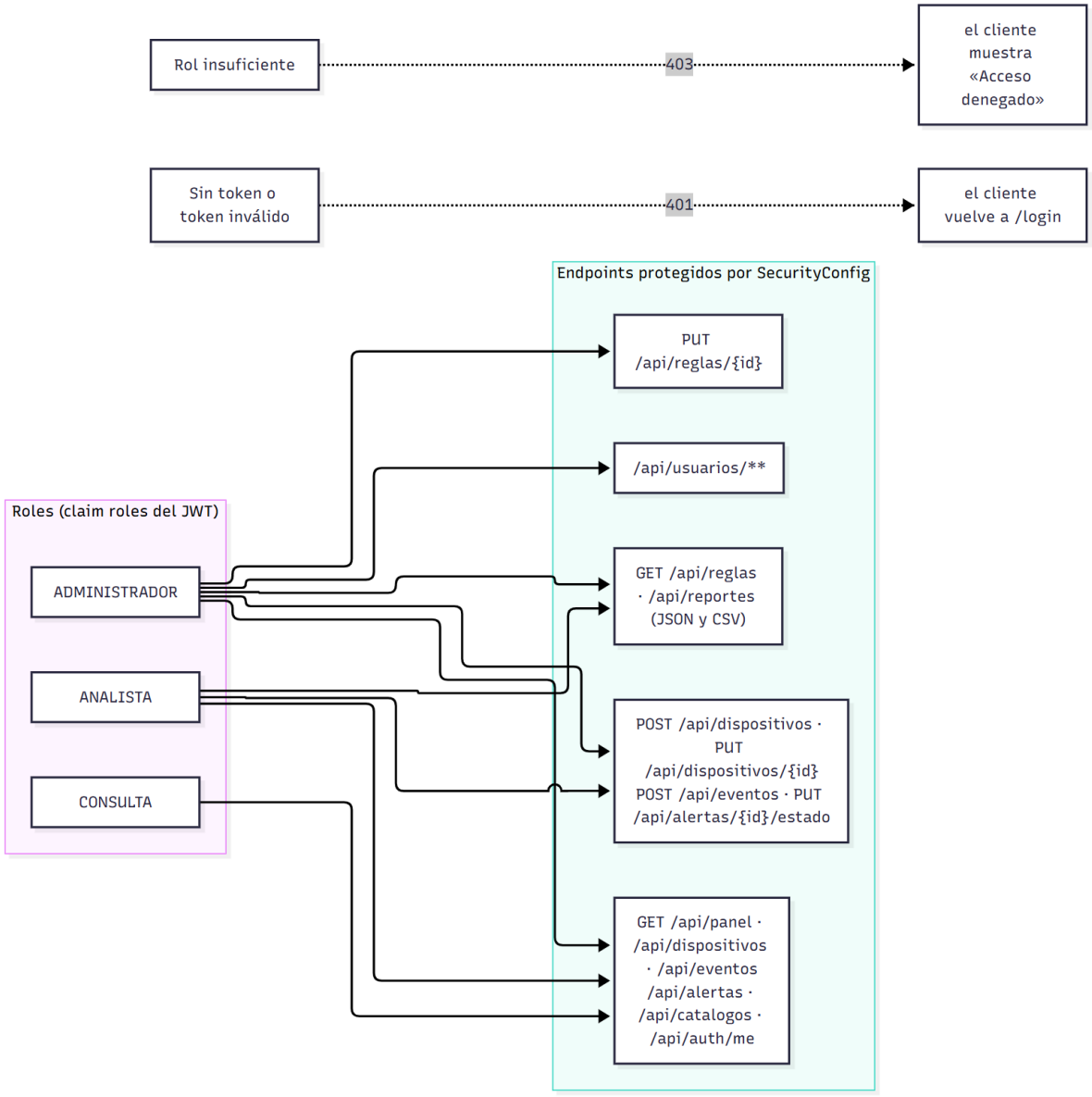
Matriz de permisos por endpoint de la API (tomada de SecurityConfig)

Endpoint	Administrador	Analista	Consulta
`POST /api/auth/login`, `/error`	Público	Público	Público
`GET /api/auth/me`, `/api/panel`, `/api/catalogos`	Sí	Sí	Sí
`GET /api/dispositivos`, `/api/eventos`, `/api/alertas`, `/api/alertas/{id}`	Sí	Sí	Sí
`POST /api/dispositivos`, `PUT /api/dispositivos/{id}`	Sí	Sí	No
`POST /api/eventos`	Sí	Sí	No
`PUT /api/alertas/{id}/estado`	Sí	Sí	No
`GET /api/reglas`, `/api/reglas/{id}`	Sí	Sí	No
`PUT /api/reglas/{id}`	Sí	No	No
`/api/reportes/**` (consulta y CSV)	Sí	Sí	No
`/api/usuarios/**`	Sí	No	No

Una petición sin token, con un token vencido o con la firma alterada recibe 401, y el cliente vuelve a la pantalla de login. Con un token válido pero sin el rol necesario la API responde 403. El cliente aplica los mismos permisos a sus rutas (`RutaProtegida.jsx`): oculta las opciones que el rol no puede usar y, si se abre la dirección a mano, muestra «Acceso denegado». La protección real está en la API; la del cliente solo evita mostrar pantallas inútiles.

Figura 1

Permisos por rol



Nota. Permisos por rol sobre los endpoints de la API según `SecurityConfig`. Elaboración propia.

8. Diagramas correspondientes al proyecto

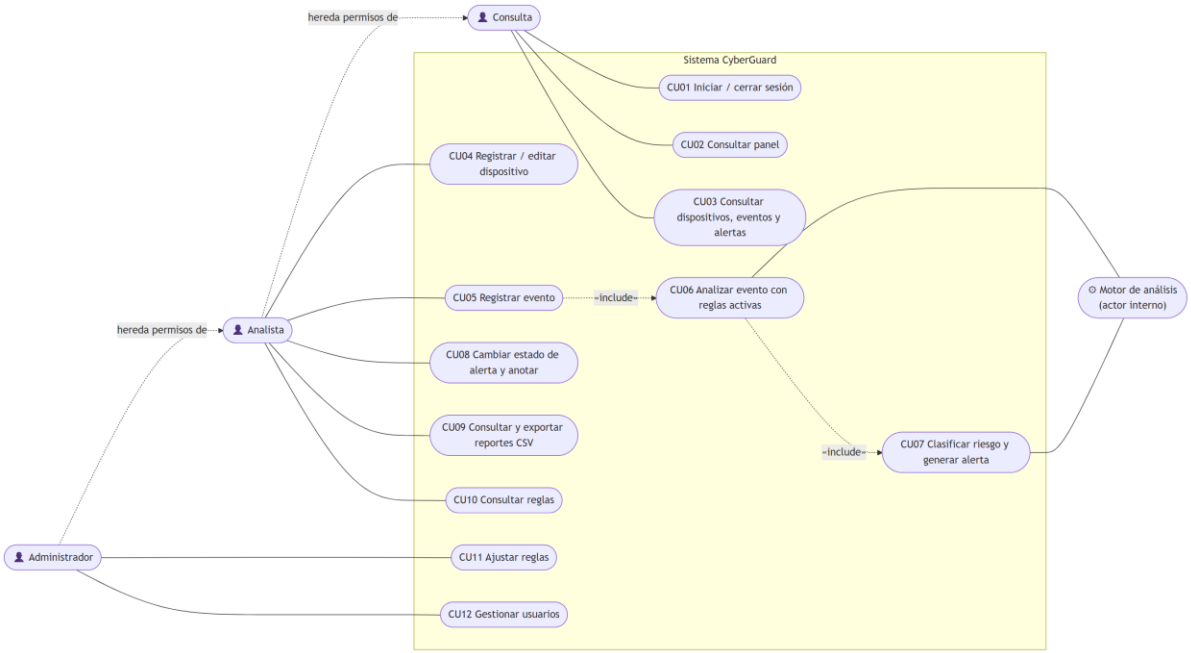
Los diagramas siguen la notación UML 2.5 (Object Management Group [OMG], 2017) y el modelo entidad-relación (Chen, 1976). Se elaboraron con Mermaid (Mermaid, s. f.) a partir del código actual. Las imágenes están en `Docs/img/` y sus fuentes editables en `Docs/img/fuentes/` (rutas del repositorio en GitHub).

8.1 Diagrama de casos de uso

Muestra qué puede hacer cada actor. Registrar un evento incluye su análisis, y el análisis incluye la clasificación y la creación de la alerta.

Figura 2

Diagrama de casos de uso



Nota. Diagrama de casos de uso de CyberGuard. Elaboración propia.

Tabla 6

Diagrama de casos de uso

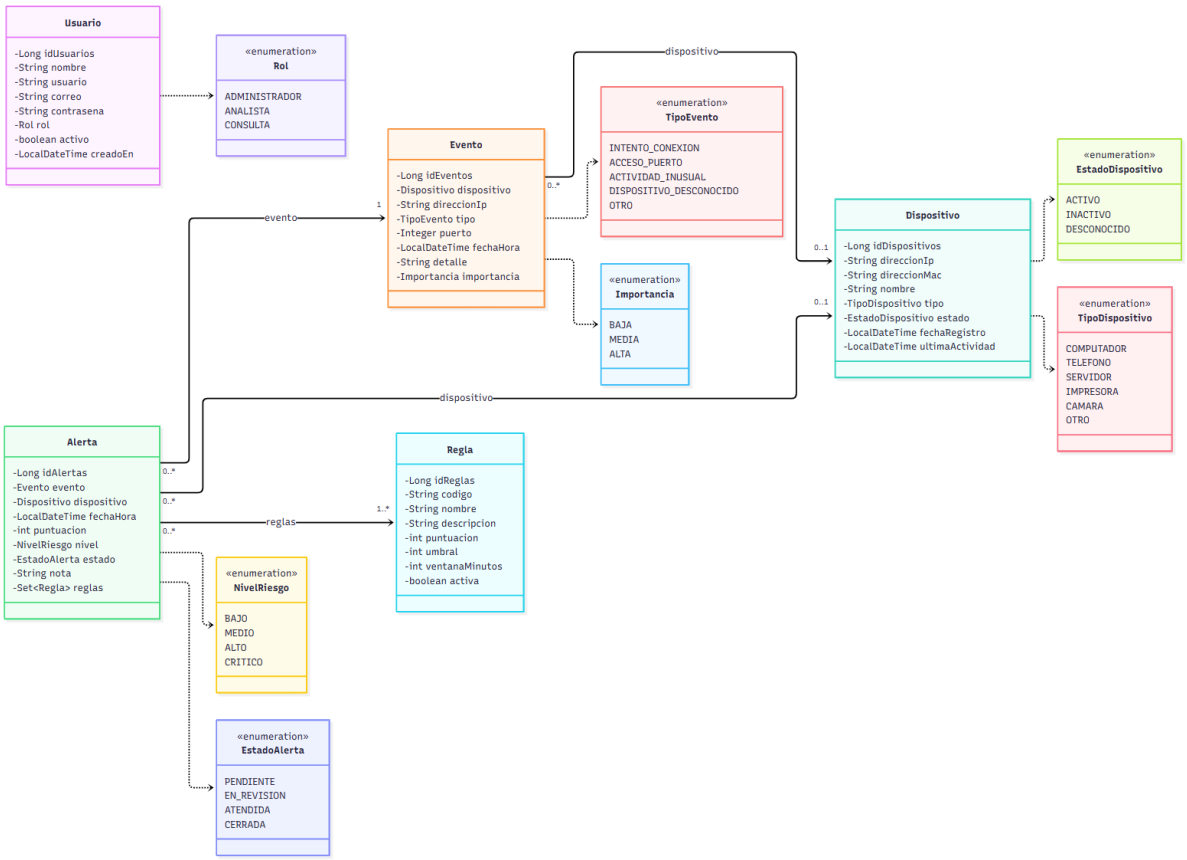
Caso de uso	Actor principal	Descripción breve
CU01 Iniciar / cerrar sesión	Todos	Ingreso con usuario y contraseña, que entrega un token JWT; al salir, el cliente borra el token.
CU02 Consultar panel	Todos	Ver indicadores y actividad reciente.
CU03 Consultar dispositivos, eventos y alertas	Todos	Listas con filtros y detalle de alerta.
CU04 Registrar / editar dispositivo	Analista	Alta y edición del inventario.
CU05 Registrar evento	Analista	Anotar un evento; dispara CU06.
CU06 Analizar evento	Motor de análisis	Evaluar las reglas activas con el historial de la IP.
CU07 Clasificar riesgo y generar alerta	Motor de análisis	Sumar puntos, asignar nivel y crear la alerta.
CU08 Cambiar estado de alerta	Analista	Pasar la alerta por los estados y dejar nota.
CU09 Consultar y exportar reportes	Analista	Filtrar y descargar CSV.
CU10 Consultar reglas	Analista	Ver el catálogo de reglas.
CU11 Ajustar reglas	Administrador	Cambiar puntuación, umbral, ventana y activación.
CU12 Gestionar usuarios	Administrador	Crear, editar y desactivar cuentas.

8.2 Diagrama de clases del dominio

Representa las cinco entidades JPA del paquete `domain` y las enumeraciones de `domain/enums`.

Figura 3

Diagrama de clases del dominio



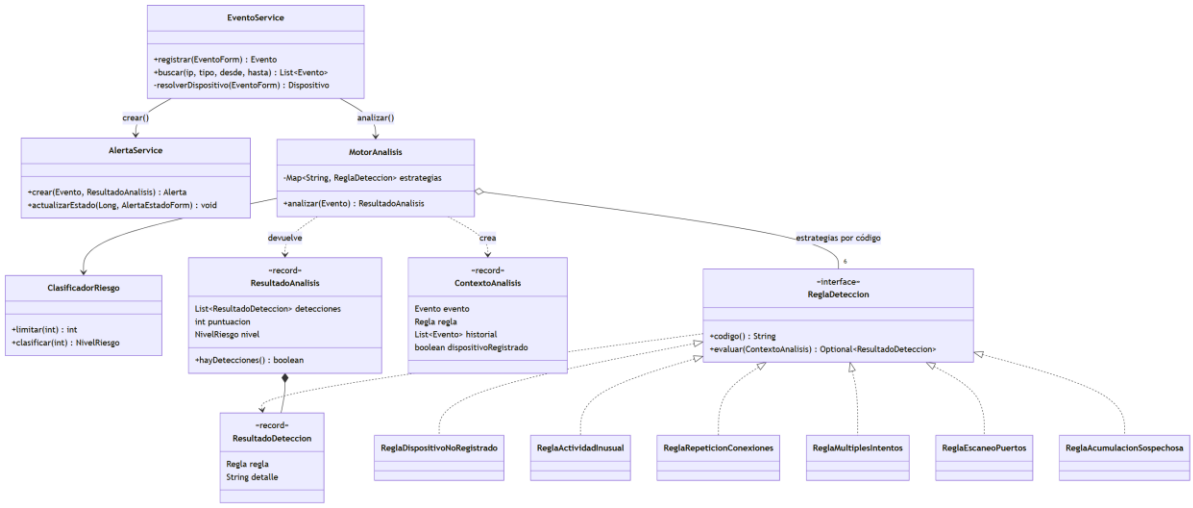
Nota. Diagrama de clases del dominio. Elaboración propia a partir de `com.cun.cyberguard.domain`.

8.3 Diagrama de clases del motor de análisis

Muestra el patrón estrategia: `MotorAnalisis` guarda un mapa `código → ReglaDeteccion` con las seis implementaciones que Spring descubre automáticamente.

Figura 4

Diagrama de clases del motor de análisis

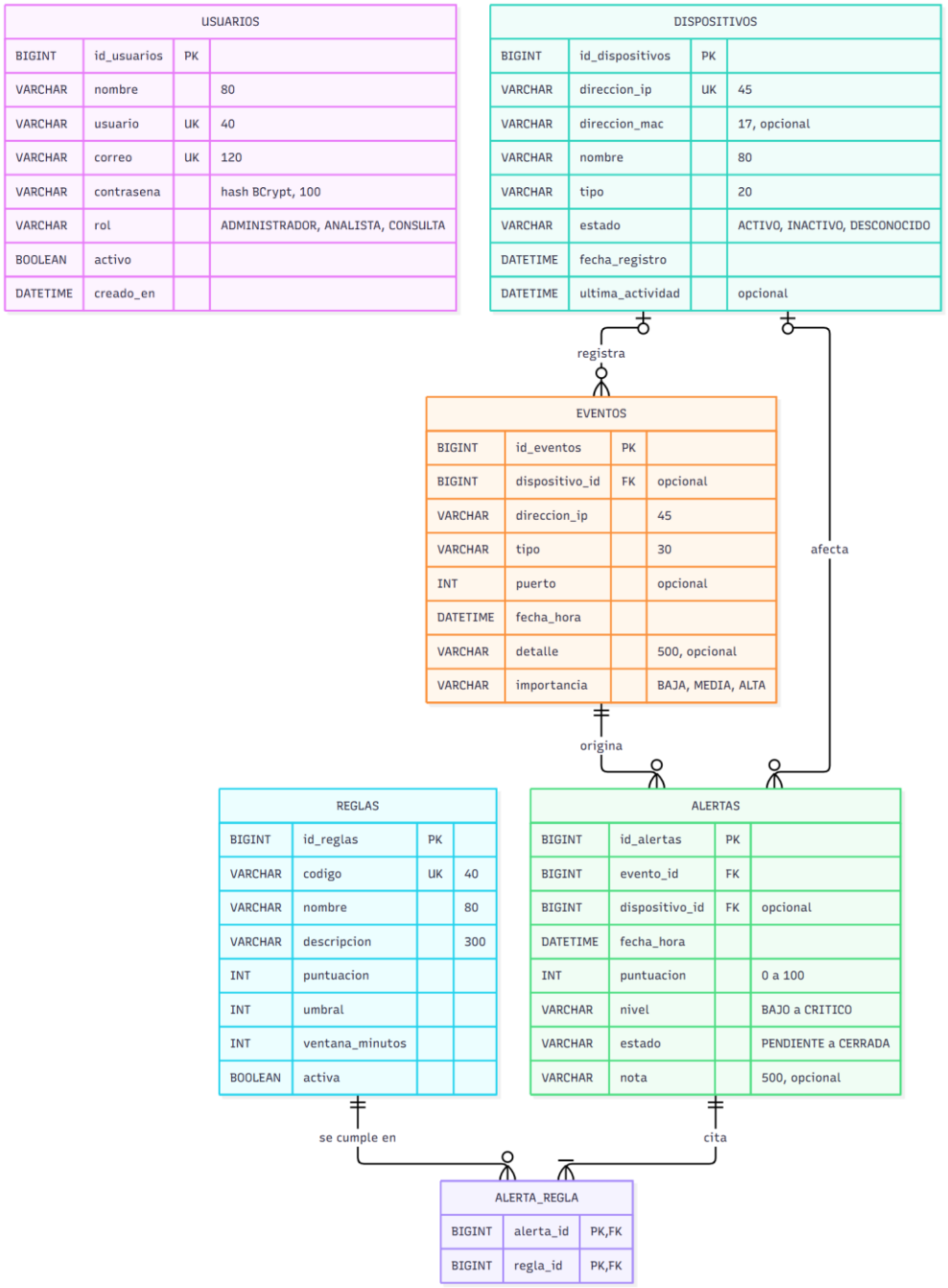


Nota. Diagrama de clases del motor de análisis. Elaboración propia a partir de `com.cun.cyberguard.analysis`.

8.4 Modelo entidad-relación

Figura 5

Modelo entidad-relación



Nota. Modelo entidad-relación de la base `cyberguard`. Elaboración propia.

Tabla 7

Modelo entidad-relación

Desde	Hacia	Cardinalidad	Columna	Observación
`eventos`	`dispositivos`	Muchos a uno, opcional	`dispositivo_id`	Un evento puede hablar de una IP que no está en el inventario.
`alertas`	`eventos`	Muchos a uno, obligatoria	`evento_id`	Toda alerta nace de un evento.
`alertas`	`dispositivos`	Muchos a uno, opcional	`dispositivo_id`	Se copia del evento al crear la alerta.
`alerta_regla`	`alertas` y `reglas`	Muchos a muchos	`alerta_id`, `regla_id`	Una alerta cita todas las reglas que se cumplieron.

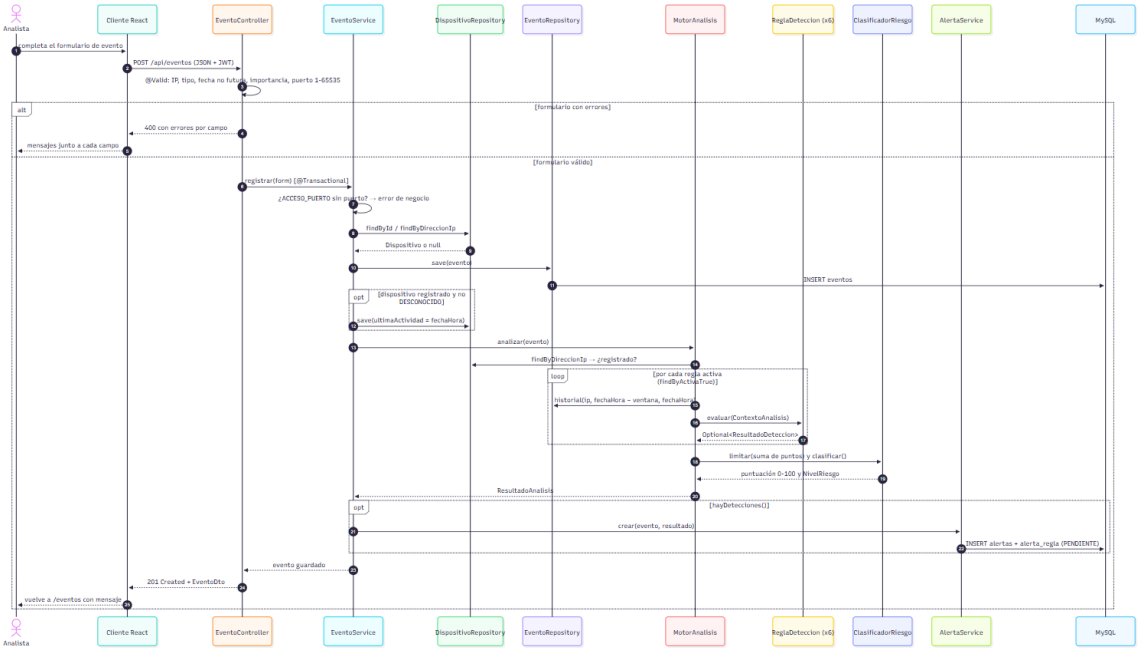
La tabla `usuarios` no se relaciona con las demás: el sistema no guarda todavía quién registró un evento o cambió una alerta (ver sección 17).

8.5 Diagrama de secuencia: registrar un evento

Es el flujo principal del sistema. El cliente envía el evento en JSON con el token, y todo el trabajo del servidor ocurre dentro de una transacción de `EventoService.registrar`.

Figura 6

Diagrama de secuencia del registro de un evento



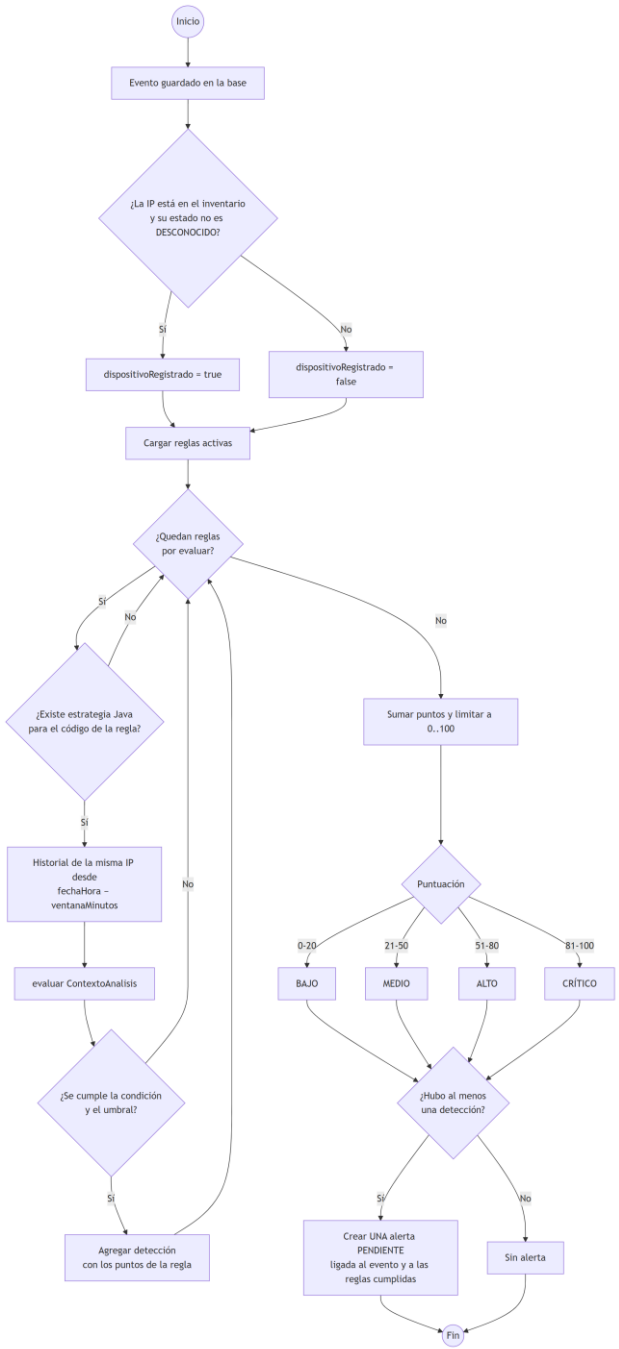
Nota. Diagrama de secuencia del registro y análisis de un evento. Elaboración propia.

8.6 Diagrama de actividades del análisis

Detalla la lógica de `MotorAnalisis.analizar` y `ClasificadorRiesgo`.

Figura 7

Diagrama de actividades del análisis

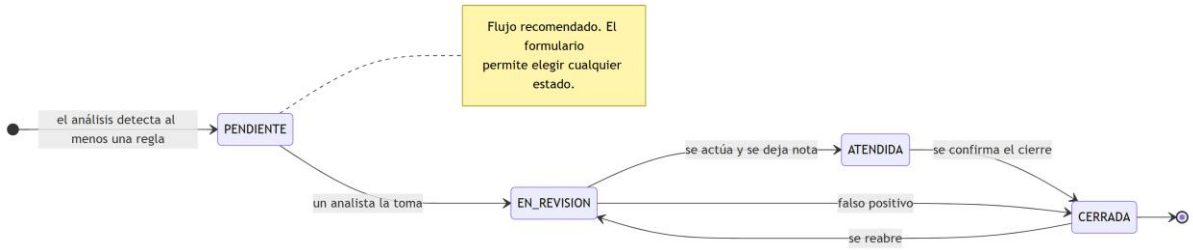


Nota. Diagrama de actividades del motor de análisis. Elaboración propia.

8.7 Diagrama de estados de la alerta

Figura 8

Diagrama de estados de la alerta



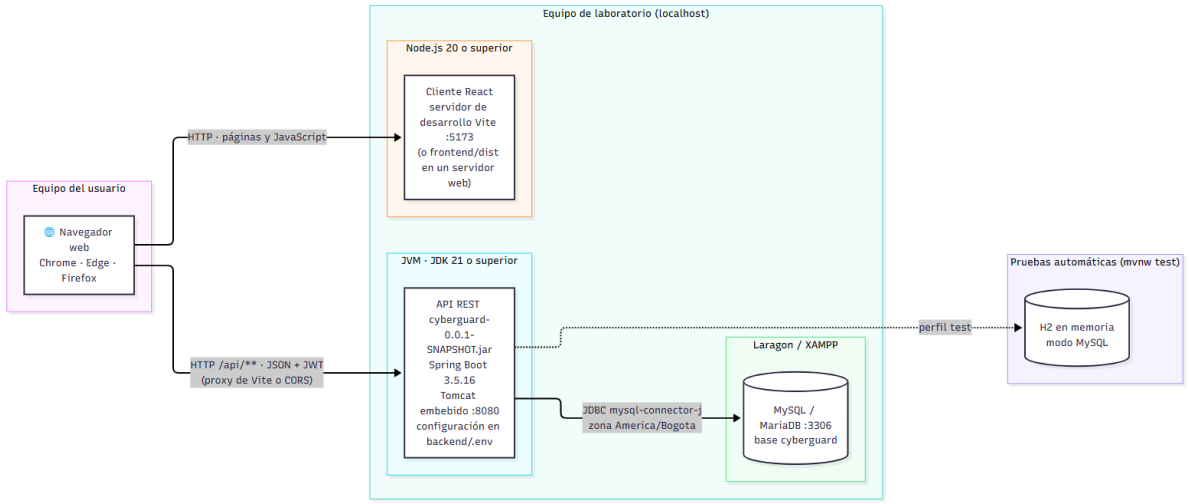
Nota. Estados de seguimiento de una alerta (enum `EstadoAlerta`). Elaboración propia.

El diagrama muestra el flujo recomendado. En la versión actual el formulario permite elegir cualquier estado; restringir las transiciones queda como mejora para la tercera entrega.

8.8 Diagrama de despliegue

Figura 9

Diagrama de despliegue



Nota. Diagrama de despliegue en el entorno de laboratorio. Elaboración propia.

Se ejecutan dos procesos: el cliente React (servidor de Vite en el puerto 5173 durante el desarrollo, o los archivos de `frontend/dist` en cualquier servidor web) y la API en el puerto 8080. En desarrollo, Vite reenvía las peticiones `/api` a la API, así que el navegador no necesita CORS; si el cliente se publica en otra dirección, su origen se agrega a `CYBERGUARD_CORS_ORIGENES`.

9. Diseño de la solución

9.1 Flujo general

```
Registro del evento → se guarda → se analizan las reglas activas
        → se suman las puntuaciones → se clasifica el riesgo
        → si hubo detecciones, se crea una alerta → se consulta en el panel
```

El análisis ocurre en el momento del registro. Consultar listas no vuelve a analizar. La puntuación y el nivel se guardan en la alerta: son el resultado de ese análisis y un cambio posterior de la regla no los reescribe.

9.2 Modelo de datos

La base `cyberguard` tiene cinco tablas principales más una tabla de unión. Hibernate las crea y actualiza al arrancar (`ddl-auto=update`). Los identificadores son `BIGINT` autoincrementales con el nombre `id_<tabla>` (por ejemplo `id_eventos`) y los enums se guardan como texto (`EnumType.STRING`), de modo que reordenar un enum en Java no altera los datos guardados.

Tabla 8

Modelo de datos

Tabla	Contenido principal
`usuarios`	Nombre, usuario único, correo único, hash BCrypt, rol, activo, fecha de creación.
`dispositivos`	IP única, MAC, nombre, tipo (computador, teléfono, servidor, impresora, cámara, otro), estado (activo, inactivo, desconocido), fecha de registro, última actividad.

Tabla	Contenido principal
`eventos`	Dispositivo opcional, IP, tipo (intento de conexión, acceso a puerto, actividad inusual, dispositivo desconocido, otro), puerto opcional, fecha y hora, detalle, importancia (baja, media, alta).
`reglas`	Código único, nombre, descripción, puntuación, umbral, ventana en minutos, activa.
`alertas`	Evento, dispositivo opcional, fecha, puntuación (0 a 100), nivel, estado y nota.
`alerta_regla`	Unión muchos a muchos entre alertas y reglas.

El detalle columna por columna está en `Docs/base-de-datos.md`.

9.3 Diseño del motor de reglas

El motor separa **qué se evalúa** de **con qué números**:

- **La condición vive en Java.** Cada regla es una clase que implementa la interfaz `ReglaDeteccion` (métodos `codigo()` y `evaluar(ContextoAnalisis)`). Es una aplicación del patrón estrategia (Gamma et al., 1994).
- **Los números viven en la base.** La fila de la tabla `reglas` guarda puntuación, umbral, ventana y si está activa. El administrador los ajusta desde la pantalla sin tocar código.
- **Un mapa une las dos partes.** Spring inyecta en `MotorAnalisis` la lista de todas las clases `@Component` que implementan `ReglaDeteccion`, y el motor las guarda en un `HashMap<String, ReglaDeteccion>` por código. Si una fila no tiene clase, se ignora.

Cada evaluación recibe un `ContextoAnalisis` (un *record* de Java) con el evento nuevo, la regla con sus parámetros, el historial reciente de la misma IP dentro de la ventana y si esa IP pertenece a un dispositivo registrado que no esté marcado como desconocido. La ventana va desde la fecha del evento menos los minutos de la regla hasta la fecha del evento: un evento registrado con una fecha pasada no cuenta los que ocurrieron después. El historial incluye el evento recién guardado; por eso un umbral de 5 significa «cinco eventos en la ventana, contando el actual».

Tabla 9

Reglas implementadas (valores iniciales cargados por DataInitializer)

Código	Clase	Se cumple cuando…	Puntos	Umbral	Ventana
`DISPOSITIVO_NO_REGISTRADO`	`ReglaDispositivoNoRegistrado`	La IP no está en el inventario o el equipo está como desconocido.	10	1	0 min
`ACTIVIDAD_INUSUAL`	`ReglaActividadInusual`	El tipo del evento es «actividad inusual».	10	1	60 min
`REPETICION_CONEXIONES`	`ReglaRepeticionConexiones`	La misma IP repite el mismo tipo de evento al menos *umbral* veces.	15	5	10 min
`MULTIPLES_INTENTOS`	`ReglaMultiplesIntentos`	El evento es un intento de conexión y la IP acumula al menos *umbral* intentos.	20	8	5 min
`ESCANEO_PUERTOS`	`ReglaEscaneoPuertos`	El evento tiene puerto y la IP tocó al menos *umbral* puertos distintos.	25	5	2 min
`ACUMULACION_SOSPECHOSA`	`ReglaAcumulacionSospechosa`	El evento es de importancia alta y la IP acumula al menos *umbral* eventos de importancia alta.	30	4	30 min

Las estructuras de datos usadas son un `HashMap` para ubicar la estrategia por código en tiempo constante, listas para el historial, *streams* con `filter`, `distinct` y `count` para contar intentos o puertos distintos, y un `LinkedHashSet` para guardar las reglas de la alerta sin duplicados y en el orden en que se cumplieron.

9.4 Clasificación de riesgo

`ClasificadorRiesgo` suma los puntos de todas las reglas cumplidas, limita el resultado entre 0 y 100 y asigna el nivel:

Tabla 10

Clasificación de riesgo

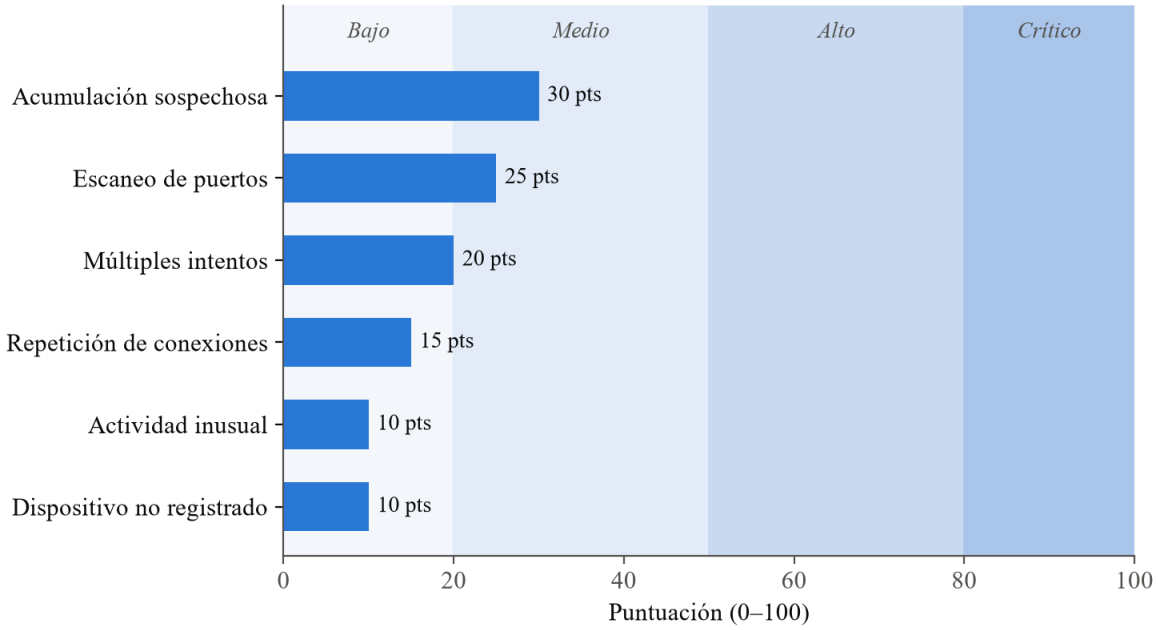
Puntuación	Nivel
0 a 20	Bajo
21 a 50	Medio

Puntuación	Nivel
51 a 80	Alto
81 a 100	Crítico

Una alerta alta o crítica es una señal para que una persona la revise, no un dictamen de que hubo un ataque. La Figura 10 compara la puntuación inicial de cada regla con los cortes de nivel: ninguna regla por sí sola supera los 30 puntos, así que los niveles alto y crítico solo se alcanzan cuando varias reglas se cumplen en el mismo evento.

Figura 10

Puntuación por regla y cortes de riesgo



Nota. Puntuación inicial de cada regla de detección frente a los cortes de nivel de riesgo de `ClasificadorRiesgo`. Elaboración propia a partir de `DataInitializer`.

9.5 Decisiones de diseño

Tabla 11

Decisiones de diseño

Decisión	Motivo
Una alerta por evento analizado	Si saltan tres reglas a la vez, queda una sola alerta con la suma y la lista de reglas. El seguimiento tiene un solo lugar.
Formularios separados de las entidades (`web/form`)	La entidad JPA no se llena directamente desde el JSON de la petición. Evita, por ejemplo, que editar un usuario sobrescriba la contraseña cifrada.
API REST y cliente React separados	Cada parte se desarrolla y prueba por su lado: la API se verifica con MockMvc sin navegador y el cliente puede cambiar sin tocar la lógica. También deja la API lista para otros clientes, como una carga masiva desde un script.
Tokens JWT en lugar de sesión en el servidor	La API no guarda estado entre peticiones y no necesita CSRF, porque el token viaja en la cabecera `Authorization` y no en una cookie. El rol va dentro del token, así que cada petición se autoriza sin consultar la base.
DTO de respuesta (`web/dto`)	La API nunca devuelve entidades JPA: no expone el hash de la contraseña y evita errores de carga perezosa al convertir a JSON.
`open-in-view` desactivado	La conexión a la base no queda abierta mientras se escribe la respuesta; las lecturas ocurren en transacciones del servicio y los controladores convierten el resultado en DTO.
Datos de demostración solo con tablas vacías	Reiniciar la aplicación no duplica filas; los eventos de ejemplo pasan por `EventoService`, así que la primera visita ya muestra alertas.
CSV con punto y coma y marca BOM	Se abre bien en Excel con configuración regional en español y conserva las tildes.
Zona horaria `America/Bogota`	Las fechas de eventos y alertas coinciden con la hora local del laboratorio.

9.6 Diseño de la interfaz

El cliente es una aplicación de una sola página en React. Un componente base (`components/Layout.jsx`) dibuja la barra de navegación, con tema oscuro y tablas con etiquetas de color por nivel de riesgo. Las rutas están en `App.jsx` y cada grupo se protege por rol con `RutaProtegida.jsx`:

Tabla 12

Diseño de la interfaz

Ruta del cliente	Pantalla	Roles
`/login`	Inicio de sesión	Público
`/`	Panel con indicadores	Todos
`/dispositivos`, `/dispositivos/nuevo`, `/dispositivos/:id/editar`	Inventario y formulario	Lista: todos; formulario: analista y administrador
`/eventos`, `/eventos/nuevo`	Historial con filtros y formulario de registro	Lista: todos; registro: analista y administrador
`/alertas`, `/alertas/:id`	Lista con filtros y detalle con seguimiento	Todos; el seguimiento solo lo ven analista y administrador
`/reglas`, `/reglas/:id/editar`	Catálogo y edición de parámetros	Catálogo: analista y administrador; edición: administrador
`/reportes`	Consulta y descarga CSV	Analista y administrador
`/usuarios`, `/usuarios/nuevo`, `/usuarios/:id/editar`	Gestión de cuentas	Administrador
`/info`	Página informativa del proyecto	Todos
cualquier otra	«Página no encontrada» (`NoEncontrada.jsx`)	Todos

Las listas de opciones (tipos de evento, niveles, estados, roles) se cargan una vez desde `GET /api/catalogos`, de modo que el cliente no repite los enums de Java.

10. Tecnologías y herramientas utilizadas

La primera entrega dejaba abiertas algunas opciones («PostgreSQL o MySQL»). La tabla muestra lo que se usa en el código actual, según `pom.xml` y `application.properties`.

Tabla 13

Tecnologías y herramientas utilizadas

Categoría	Tecnología	Versión	Uso en el proyecto
Lenguaje	Java	21 (compila con JDK 21 o superior)	Todo el backend (Oracle, s. f.).

Categoría	Tecnología	Versión	Uso en el proyecto
Framework	Spring Boot	3.5.16	Configuración, servidor embebido e inyección de dependencias (VMware, s. f.).
Web	Spring MVC + Tomcat embebido	Incluido en Spring Boot	Controladores REST con respuestas JSON en el puerto 8080.
Seguridad	Spring Security + servidor de recursos OAuth2	6.x	Login, roles por endpoint y BCrypt (Spring, s. f.-a).
Tokens	JWT firmado con HS256 (Nimbus JOSE + JWT)	Incluido en `spring-boot-starter-oauth2-resource-server`	Emisión y verificación del token de sesión (Jones et al., 2015).
Configuración	spring-dotenv (`springboot3-dotenv`)	5.1.0	Lee `backend/.env` con la conexión a la base, la clave JWT, CORS y el puerto.
Validación	Jakarta Bean Validation (Hibernate Validator)	3.x	Reglas en los formularios (`@NotBlank`, `@Pattern`, `@Min`, `@Max`, `@Email`, `@Size`).
Persistencia	Spring Data JPA + Hibernate ORM	6.x	Repositorios y mapeo objeto-relacional (Spring, s. f.-b).
Base de datos	MySQL / MariaDB (XAMPP o Laragon)	MySQL 8+ / MariaDB 10+	Almacenamiento principal (Oracle Corporation, s. f.).
Driver	mysql-connector-j	Gestionado por Spring Boot	Conexión JDBC.
Base de pruebas	H2 en memoria (modo MySQL)	Gestionado por Spring Boot	Pruebas sin MySQL.
Cliente web	React + React Router	19 / 7	Interfaz de una sola página que consume la API (Meta Open Source, s. f.).
Herramientas del cliente	Vite + Node.js	Vite 8 / Node.js 20+	Servidor de desarrollo con proxy a la API y compilación a archivos estáticos (Vite, s. f.).
Estilos	CSS3	—	Hoja propia (`frontend/src/index.css`).
Pruebas	JUnit 5, Mockito, Spring Security Test, MockMvc	Incluidos en `spring-boot-starter-test`	Pruebas unitarias y de integración (JUnit Team, s. f.; Mockito, s. f.).
Construcción	Maven Wrapper	Maven 3.9.11	Compilar, probar y ejecutar sin instalar Maven.
Control de versiones	Git y GitHub	—	Historial y trabajo colaborativo.
Diagramas	Mermaid	11	Diagramas UML y ER de este documento.
Entornos de desarrollo	IntelliJ IDEA, Visual Studio Code, Cursor	—	Edición del código.
Asistentes de desarrollo	Claude Code	—	Apoyo en código y documentación, con revisión del equipo.

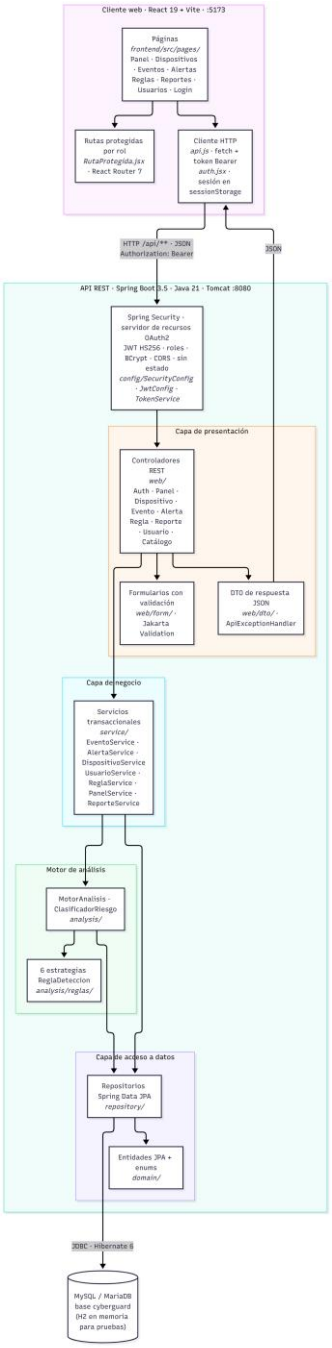
Categoría	Tecnología	Versión	Uso en el proyecto
Gestión de tareas	Trello / GitHub Projects	—	Tablero del backlog y de los sprints.

11. Arquitectura propuesta

CyberGuard está formado por dos aplicaciones. El cliente web en React corre en el navegador y llama a una API REST en Spring Boot, que está organizada en capas (Fowler, 2002). El cliente envía y recibe JSON y adjunta en cada petición el token JWT obtenido al iniciar sesión; la API no guarda sesiones. La API sigue el estilo arquitectónico REST: cada recurso (dispositivos, eventos, alertas, reglas) se identifica con una URL, se opera con los métodos estándar de HTTP y cada petición lleva toda la información necesaria para atenderla, sin estado compartido entre cliente y servidor (Fielding, 2000).

Figura 11

Arquitectura por capas



Nota. Arquitectura de CyberGuard: cliente React y API REST por capas. Elaboración propia.

Tabla 14

Arquitectura propuesta

Capa	Paquete	Responsabilidad
Cliente web	`frontend/src/`	Pantallas, rutas por rol, sesión en `sessionStorage` y llamadas a la API (`api.js`).
Seguridad	`config/`	Filtro de Spring Security, emisión y verificación de JWT (`JwtConfig`, `TokenService`), CORS, codificador BCrypt, carga de usuarios y datos de demostración.
Presentación	`web/`, `web/form/`, `web/dto/`	Recibir peticiones JSON, validar formularios, devolver DTO y traducir errores a respuestas uniformes (`ApiExceptionHandler`).
Negocio	`service/`	Reglas de negocio, transacciones y consultas que necesita cada pantalla.
Análisis	`analysis/`, `analysis/reglas/`	Evaluar reglas, sumar puntos y clasificar el riesgo.
Acceso a datos	`repository/`, `domain/`	Repositorios Spring Data, entidades JPA y enums.
Base de datos	MySQL/MariaDB	Persistencia de la información.

Cada capa llama solo a la de abajo, por inyección de dependencias en el constructor. Un controlador no escribe SQL ni decide si un escaneo es sospechoso: llama a un servicio, y el servicio llama al motor cuando hace falta. Esta separación mantiene las dependencias en una sola dirección y permite probar el motor sin base de datos, como hace `MotorAnalisisTest` con Mockito (Martin, 2017).

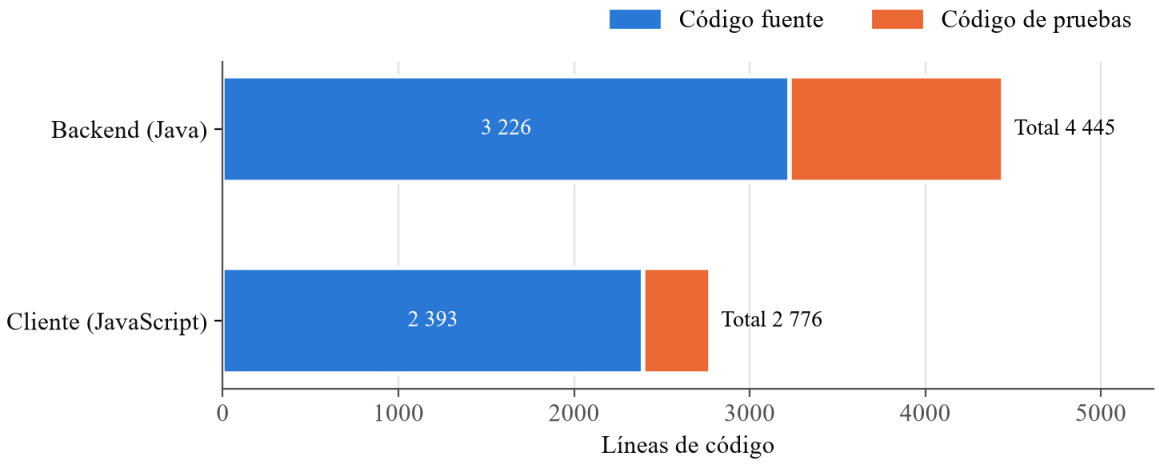
Estructura del código fuente

```
backend/src/main/java/com/cun/cyberguard
├── CyberguardApplication.java   arranque de Spring Boot
├── config/                      SecurityConfig, JwtConfig, TokenService, UsuarioDetailsService,
DataInitializer
├── web/                         9 controladores REST + ApiExceptionHandler
│   ├── form/                    5 formularios con validaciones
│   └── dto/                     10 DTO de respuesta
├── service/                     7 servicios + utilidad de fechas + excepción de recurso no
encontrado
├── analysis/                    MotorAnalisis, ClasificadorRiesgo, contratos y records
│   └── reglas/                  6 reglas de detección
├── domain/                      5 entidades JPA
│   └── enums/                   7 enumeraciones
└── repository/                  5 repositorios Spring Data
backend/src/main/resources
└── application.properties       lee sus valores de backend/.env
backend/src/test                 13 clases de prueba + application-test.properties
frontend/src
├── main.jsx, App.jsx            arranque y rutas
├── api.js, auth.jsx             cliente HTTP con token y contexto de sesión
├── catalogos.jsx, util.js       catálogos de la API y utilidades
├── components/                  Layout, RutaProtegida, controles y hook de formularios
├── pages/                       15 páginas
└── *.test.js(x), test/          suite Vitest (24 pruebas)
```

En total hay 70 archivos Java en `backend/src/main` y unas 7 200 líneas de código entre el backend (Java, fuente y pruebas) y el cliente (JavaScript). La Figura 12 muestra su reparto: cerca de una cuarta parte del código del backend corresponde a pruebas automáticas.

Figura 12

Líneas de código por componente



Nota. Líneas de código del backend y del cliente, separadas en código fuente y código de pruebas. Elaboración propia a partir del repositorio (5 de noviembre de 2026).

12. Descripción de las funcionalidades desarrolladas

12.1 Autenticación y control de acceso

- `POST /api/auth/login` recibe usuario y contraseña y devuelve un token JWT firmado con HS256, válido por 120 minutos, con el usuario y su rol. `GET /api/auth/me` devuelve los datos del usuario del token.
- `UsuarioDetailsService` carga el usuario desde la base; si está inactivo, no puede entrar («La cuenta está inactiva»).
- Contraseñas cifradas con BCrypt (Provos y Mazières, 1999).
- La API es sin estado: cada petición lleva el token en la cabecera `Authorization: Bearer`. Sin token, con el token vencido o con la firma alterada responde 401; sin el rol necesario, 403.
- La clave de firma sale de `CYBERGUARD_JWT_SECRET` y la aplicación no arranca si tiene menos de 32 bytes.
- El cliente guarda el token en `sessionStorage`, que se borra al cerrar la pestaña. Ante un 401 cierra la sesión y vuelve al login; al pulsar «Salir» borra el token y muestra «Sesión cerrada».
- El cliente protege sus rutas por rol, muestra «Acceso denegado» si se abre una dirección sin permiso y el menú oculta las opciones que el rol no puede usar.

12.2 Gestión de usuarios (administrador)

- Lista, creación y edición de usuarios con nombre, usuario, correo, rol, contraseña y estado activo.
- Usuario y correo se guardan en minúsculas y deben ser únicos.

- Contraseña obligatoria al crear y de al menos 8 caracteres; al editar, si se deja vacía se conserva la actual.
- Protecciones: nadie puede desactivarse a sí mismo y siempre debe quedar al menos un administrador activo.

12.3 Inventario de dispositivos

- Lista ordenada por nombre y formulario de alta y edición.
- Validación de IPv4 y de MAC (`AA:BB:CC:DD:EE:FF` o con guiones); la MAC se guarda en mayúsculas.
- La IP no se puede repetir entre dispositivos.
- La última actividad se actualiza sola cuando se registra un evento del dispositivo, salvo que esté marcado como desconocido.

12.4 Registro y consulta de eventos

- Formulario con dispositivo opcional, IP, tipo, puerto (1 a 65535), fecha y hora, detalle (hasta 500 caracteres) e importancia.
- Al elegir un dispositivo, el cliente rellena la IP; la fecha se propone con la hora actual y no se aceptan fechas futuras (`@PastOrPresent` en la API y `max` en el campo de fecha).
- Si no se elige dispositivo, se busca la IP en el inventario; si no existe, el evento se guarda igual, sin dispositivo.
- El tipo «acceso a puerto» exige puerto.
- Historial filtrable por IP parcial, tipo y rango de fechas.

12.5 Motor de análisis y clasificación

- Se ejecuta automáticamente al guardar cada evento.

- Evalúa solo las reglas activas, con el historial de la misma IP dentro de la ventana de cada regla.
- Suma los puntos, los limita a 0–100 y asigna el nivel de riesgo.
- Seis reglas implementadas (sección 9.3).

12.6 Alertas y seguimiento

- Se crea una alerta pendiente cuando al menos una regla se cumple, ligada al evento, al dispositivo y a todas las reglas cumplidas.
- Lista filtrable por nivel, estado y rango de fechas.
- Detalle con fecha, IP, dispositivo, evento, puntuación, nivel, estado y las reglas activadas con sus puntos y descripción.
- El analista y el administrador cambian el estado (pendiente, en revisión, atendida, cerrada) y dejan una nota de hasta 500 caracteres.

12.7 Gestión de reglas

- Catálogo visible para analista y administrador.
- El administrador cambia la puntuación (1 a 100), el umbral (mínimo 1), la ventana (0 a 1 440 minutos) y si la regla está activa.
- Los cambios aplican a los eventos que se registren después.

12.8 Panel de información

- Indicadores: dispositivos, eventos, alertas, alertas de riesgo alto y alertas críticas.
- Últimos 8 eventos y últimas 5 alertas con acceso al detalle.

12.9 Reportes

- Consulta de eventos (por IP, tipo y fechas) y de alertas (por nivel, estado y fechas).

- Descarga en CSV separado por punto y coma, con BOM UTF-8, y con escape de comillas y saltos de línea. El cliente pide el archivo con el token y lo guarda desde el navegador, porque un enlace directo no podría enviar la cabecera de autorización.

12.10 Página informativa

- Ruta `/info`, disponible para cualquier usuario autenticado desde el enlace «Info» del menú.
- Recorre en diez secciones qué es CyberGuard, cómo fluye un evento, la arquitectura por capas, el modelo de datos con su cardinalidad y normalización, el motor de reglas, la seguridad por roles, la gobernanza del equipo y las *skills* de asistentes de IA usadas en el desarrollo.
- Los diagramas se dibujan en SVG dentro del propio cliente (`components/info/Diagramas.jsx`) y el texto vive en `components/info/contenido.js`, separado de la presentación.

12.11 Datos de demostración

`DataInitializer` crea, solo si las tablas están vacías: los tres usuarios, las seis reglas, tres dispositivos (`PC-Laboratorio` 192.168.10.11, `Servidor-Academico` 192.168.10.20, `Camara-Pasillo` 192.168.10.30) y una serie de eventos de ejemplo que activan cada tipo de regla: actividad inusual, un equipo no inventariado, ocho intentos de conexión, accesos a cinco puertos distintos y cuatro eventos de importancia alta.

13. Avance del desarrollo

13.1 Avance por fases de la planificación

Tabla 15

Avance por fases de la planificación

Fase	Actividades principales	Estado
1. Análisis	Problema, objetivos, usuarios y requerimientos	Terminada (primera entrega)
2. Diseño	Arquitectura, módulos, base de datos y diagramas	Terminada (esta entrega)
3. Desarrollo inicial	Usuarios, dispositivos y eventos	Terminada
4. Motor de análisis	Reglas y clasificación de riesgo	Terminada
5. Alertas e interfaz	Alertas, panel, consultas y reportes	Terminada
6. Integración	Integración de módulos, seguridad y datos de demostración	Terminada
6b. Separación cliente / API	API REST con JWT y cliente React	Terminada
7. Pruebas	Pruebas de componentes y corrección de errores	En curso
8. Documentación	Documento del proyecto, guía, instalación y base de datos	En curso

13.2 Avance por módulo

Tabla 16

Avance por módulo

Módulo	Estado	Observación
Autenticación y usuarios	Completo	Falta recuperación de contraseña (fuera del alcance inicial).
Dispositivos	Completo	No hay eliminación; se usa el estado inactivo.
Eventos	Completo	Registro manual; falta carga masiva.
Motor de análisis	Completo	Seis reglas funcionando.
Clasificación de riesgo	Completo	Probado en sus límites.
Alertas	Completo	Transiciones de estado libres.
Reglas	Completo	Solo se editan parámetros, no se crean reglas nuevas desde la interfaz.
Panel	Completo	Indicadores numéricos; sin gráficas.
Reportes	Completo	Solo CSV.

Módulo	Estado	Observación
Cliente web (React)	Completo	15 páginas, incluida la página informativa; 24 pruebas con Vitest (api, auth, rutas, login y UI).
Pruebas automáticas	En curso	62 del backend y 24 del cliente; falta medir cobertura con JaCoCo y ampliar pantallas de negocio del cliente.

Los 13 requerimientos funcionales y los 8 no funcionales tienen implementación en el código. El trabajo pendiente se concentra en cobertura, trazabilidad y funciones complementarias (sección 17).

13.3 Historial del repositorio

Tabla 17

Historial del repositorio

Commit	Fecha	Descripción
`d1844c7`	2026-09-28	Versión inicial de la aplicación
`cd061a4`	2026-09-28	Capturas de pantalla del recorrido por la interfaz
`984e765`	2026-09-28	Organización de la documentación y guía de instalación
`729d389`	2026-09-28	Documentación del esquema de base de datos
`78aa6cb`	2026-09-30	Documento de la segunda entrega
`5d875b9`	2026-09-30	Mejoras de nulabilidad y documentación
`299aacf`	2026-10-05	Separación en API REST con JWT y cliente React
`8dd6268`	2026-11-05	Alineación de identificadores, análisis y configuración (`.env`)
`e7e22fb`	2026-11-05	*Skills* de asistentes de IA y su documentación
`007ed7b`	2026-11-05	Página informativa y ampliación de pruebas (62 backend, 24 cliente)

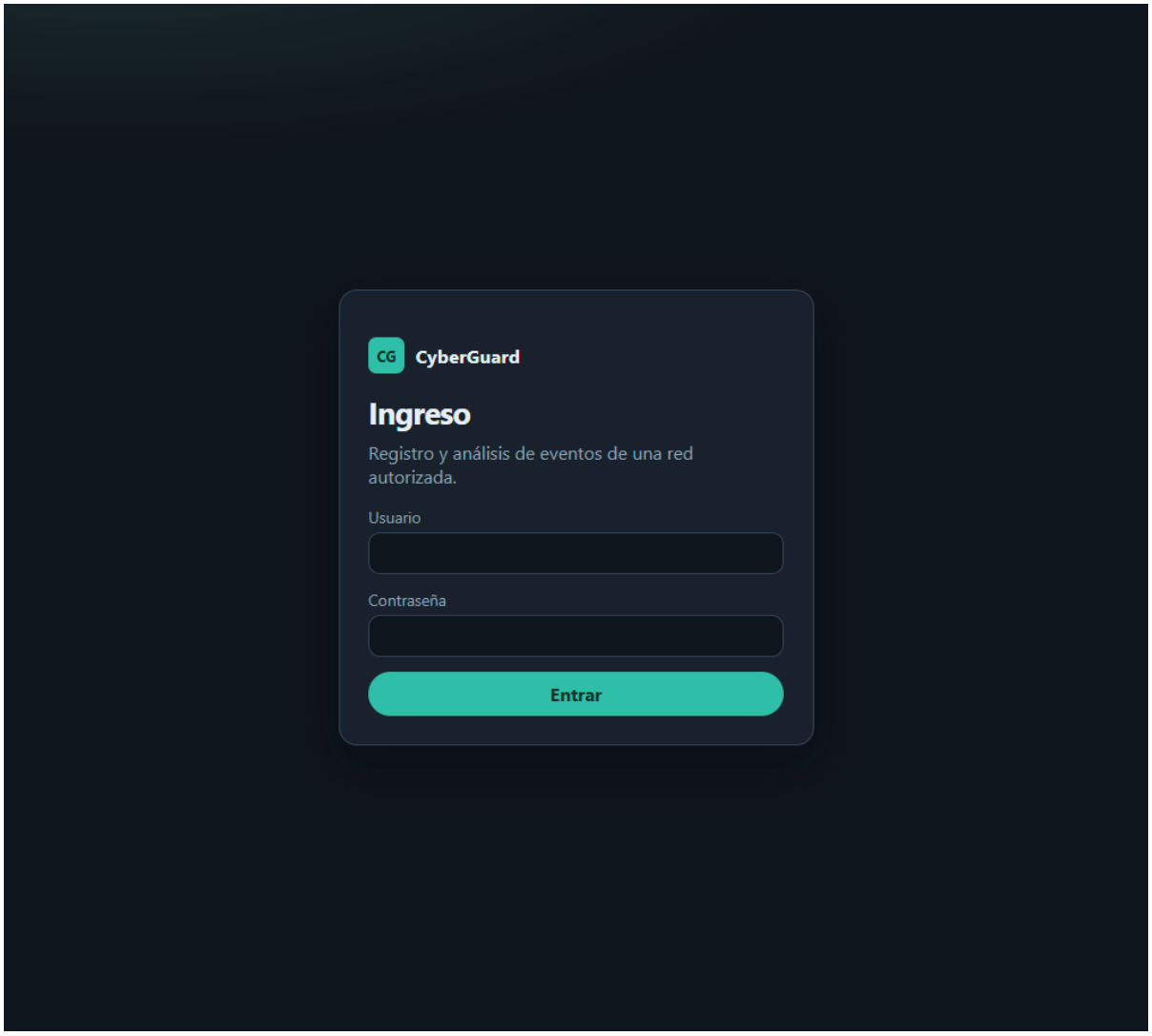
14. Evidencias del sistema

Las capturas se tomaron de la aplicación en ejecución con los datos de demostración. Están en la carpeta `screenshots/` del repositorio.

14.1 Acceso

Evidencia 1

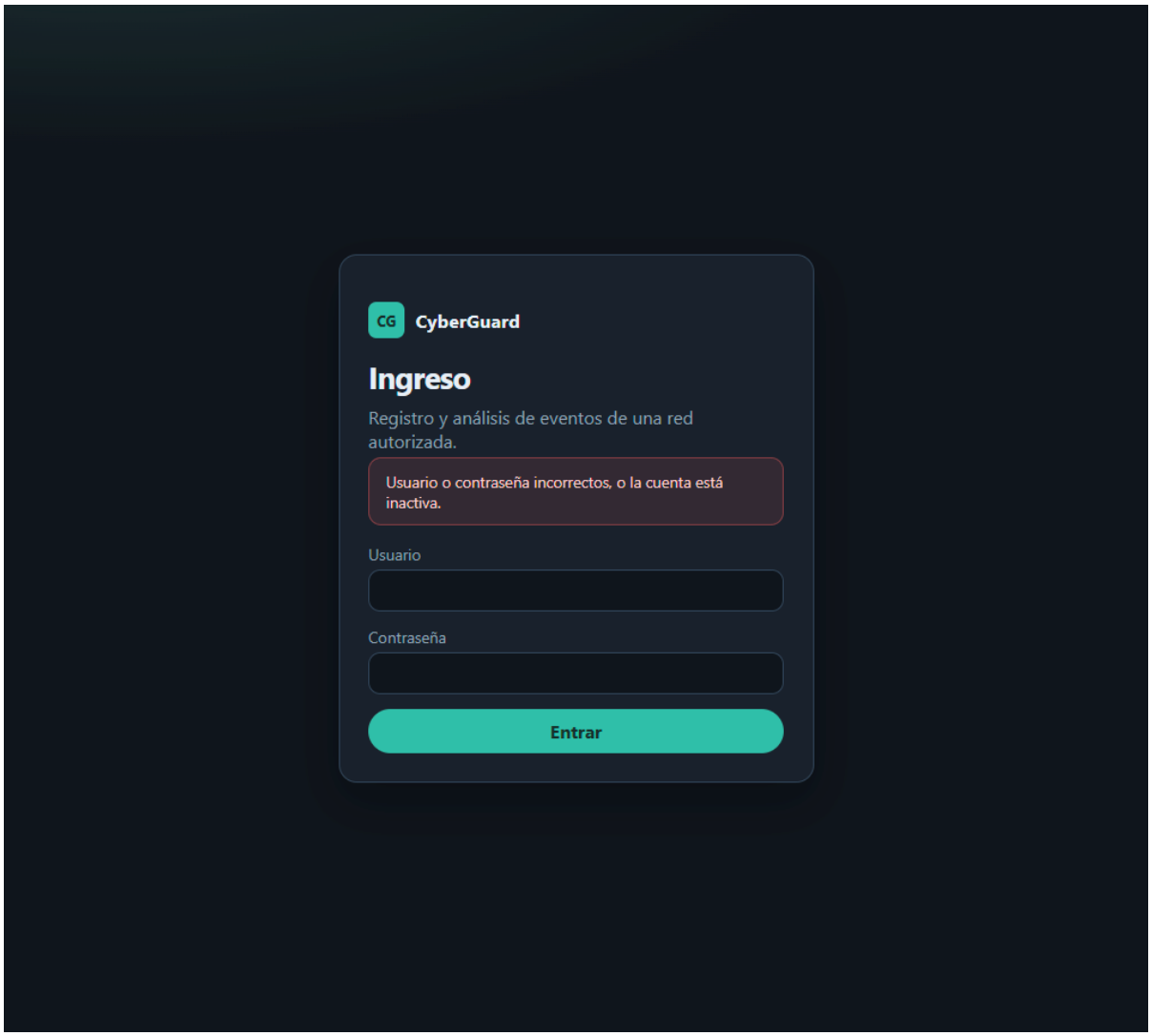
Inicio de sesión



Nota. Pantalla de inicio de sesión.

Evidencia 2

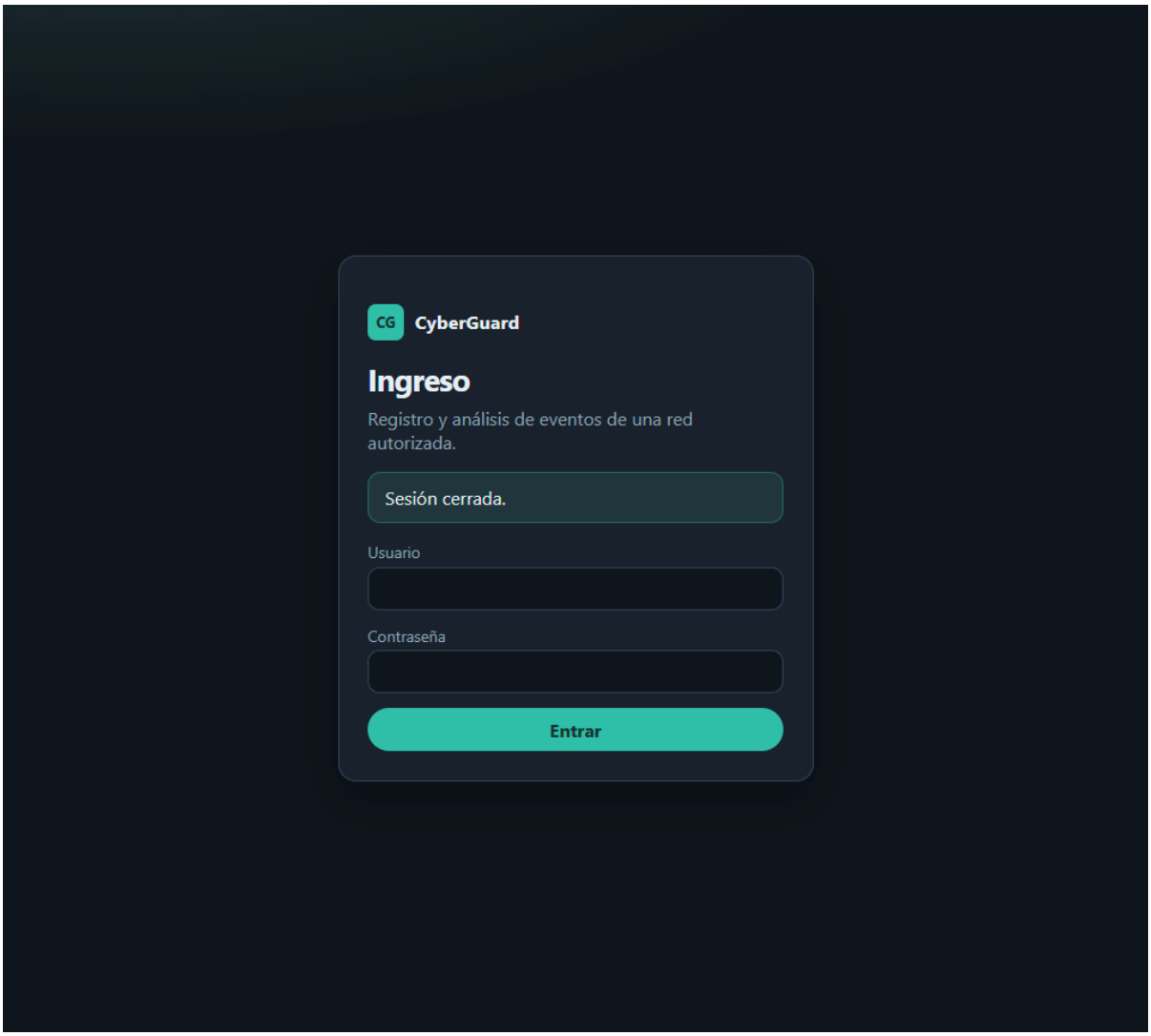
Error de credenciales



Nota. Mensaje al ingresar credenciales incorrectas.

Evidencia 3

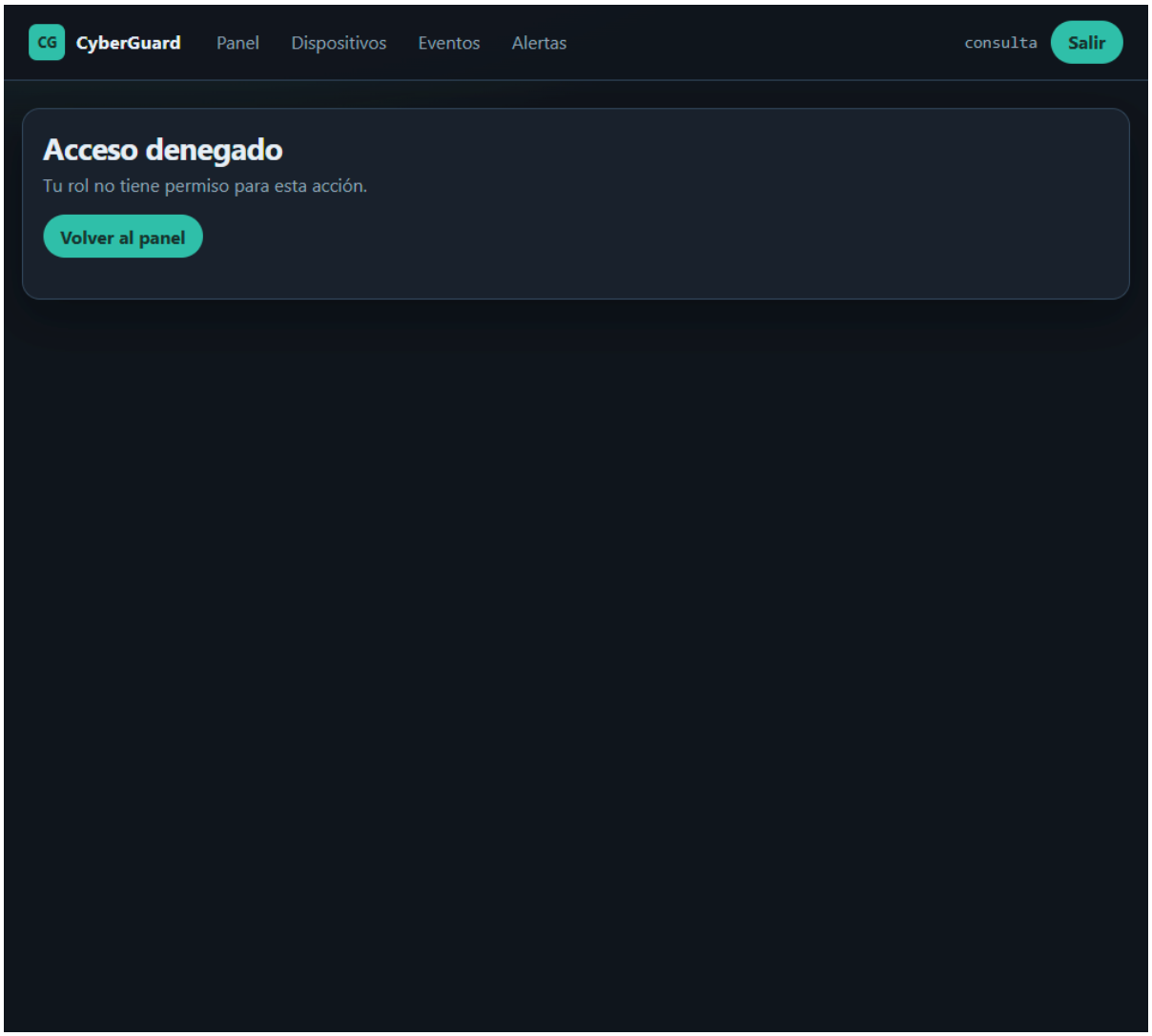
Sesión cerrada



Nota. Mensaje tras cerrar la sesión.

Evidencia 4

Acceso denegado

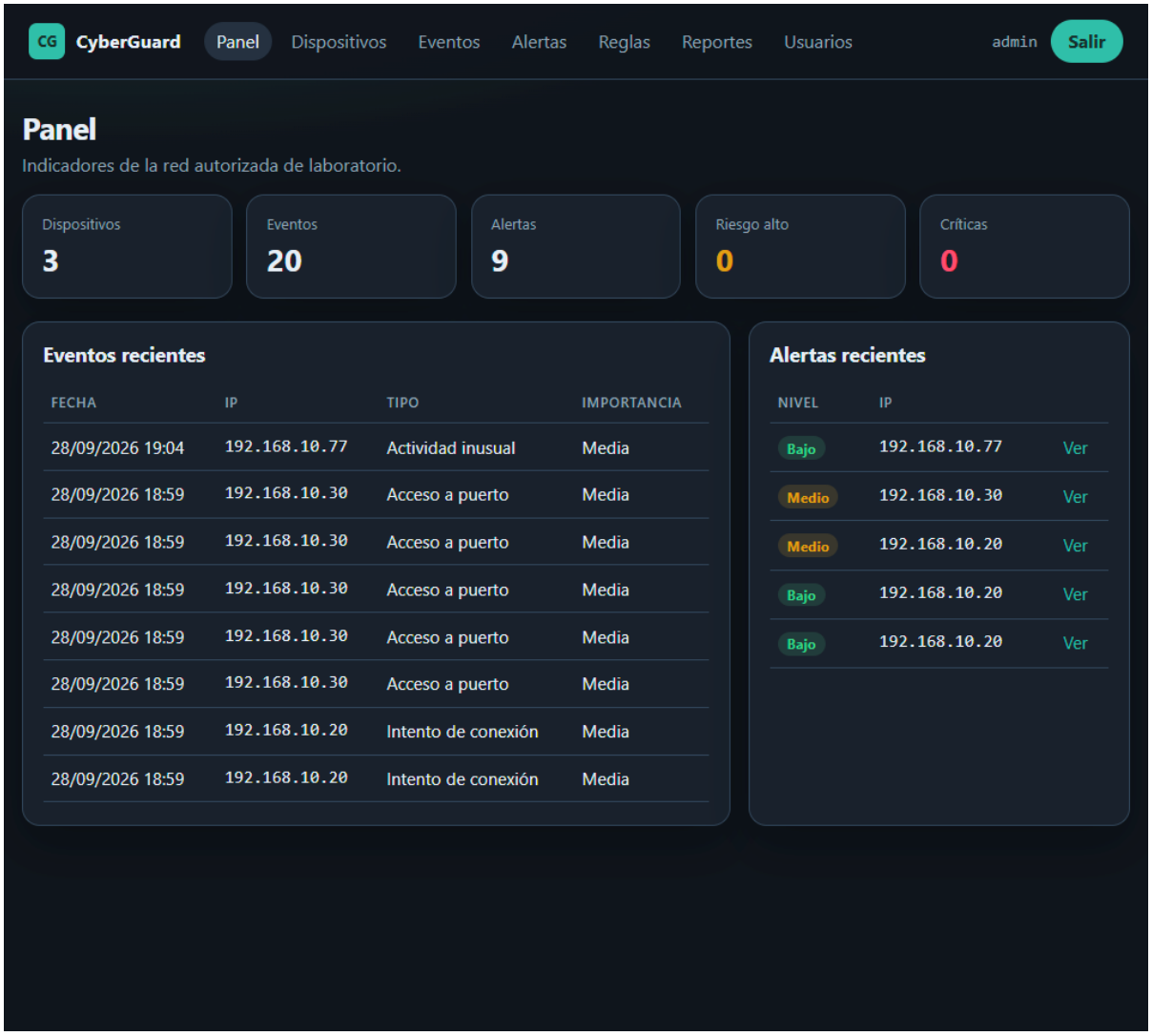


Nota. Mensaje de acceso denegado cuando el rol no tiene permiso.

14.2 Panel

Evidencia 5

Panel

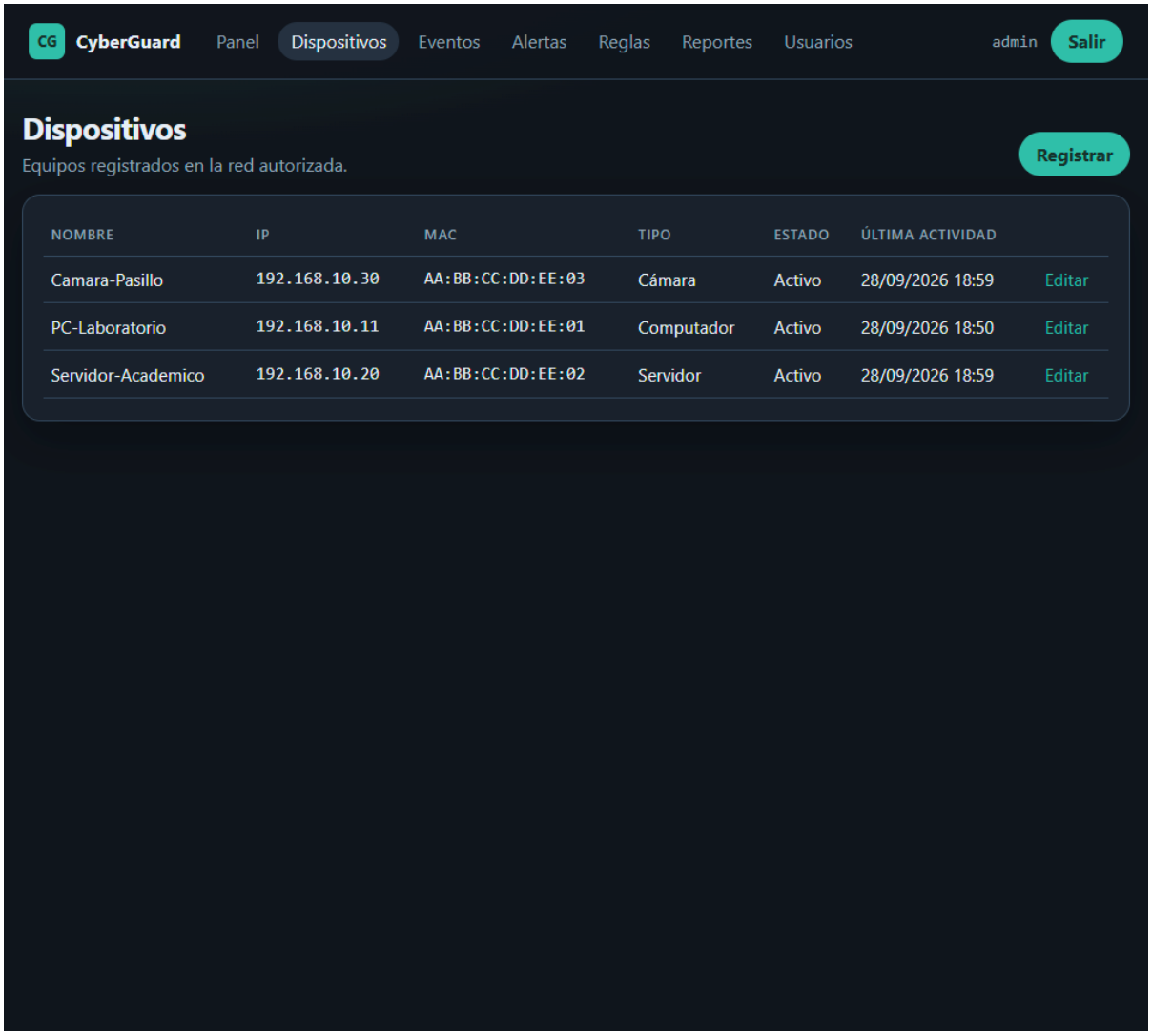


Nota. Panel del administrador con 3 dispositivos, 20 eventos y 9 alertas, eventos y alertas recientes.

14.3 Dispositivos

Evidencia 6

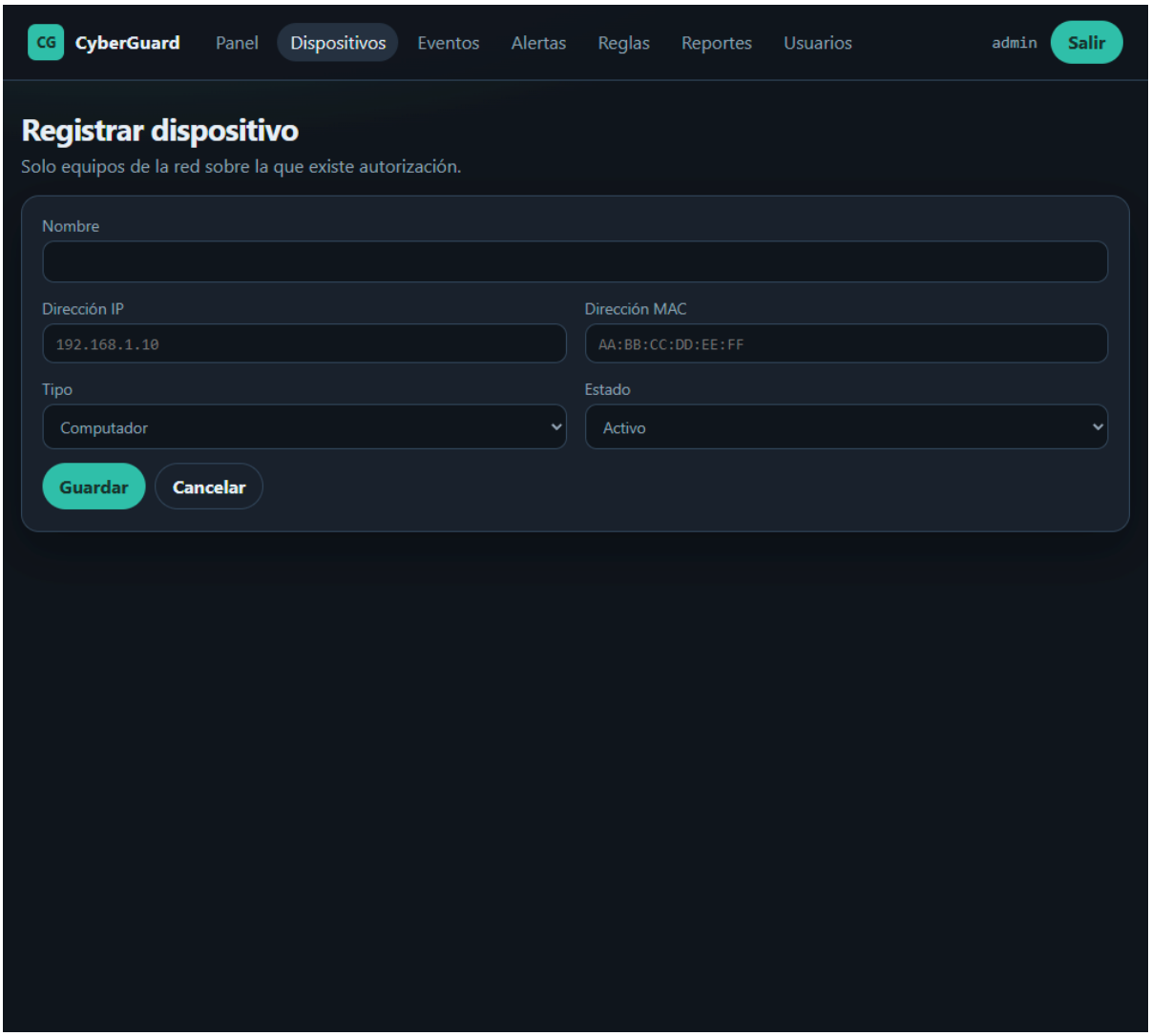
Lista de dispositivos



Nota. Inventario de dispositivos.

Evidencia 7

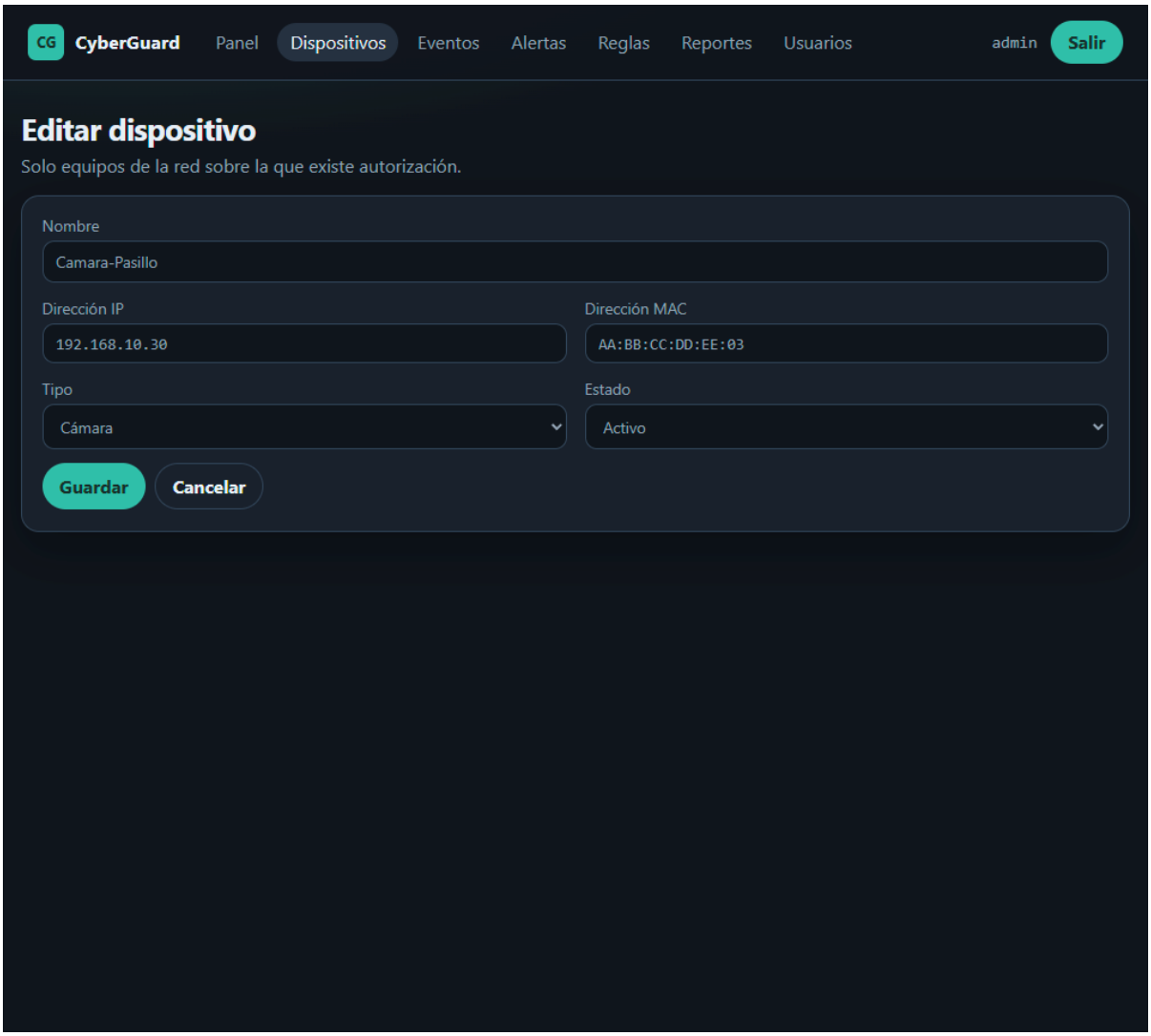
Nuevo dispositivo



Nota. Formulario de alta de dispositivo.

Evidencia 8

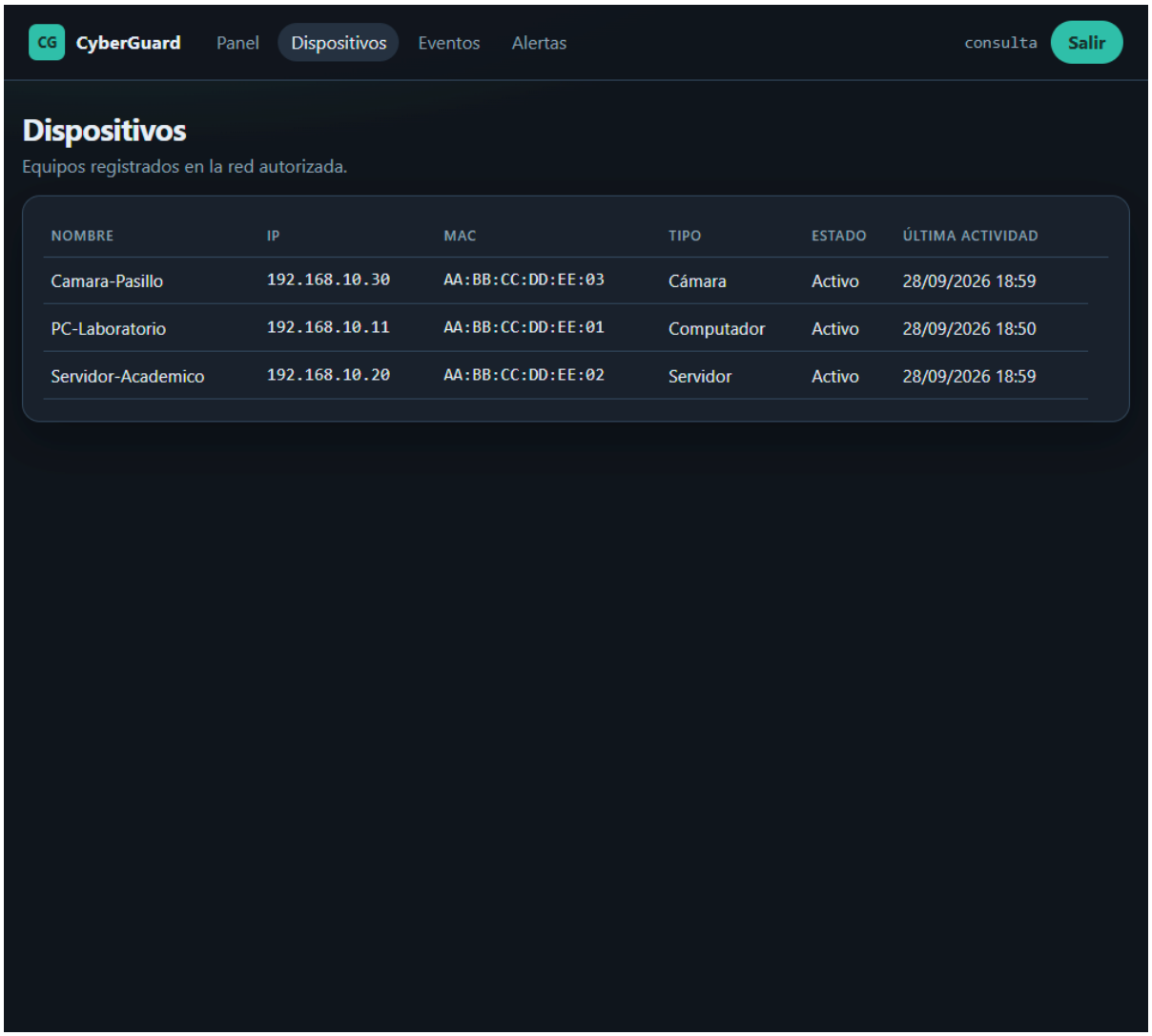
Editar dispositivo



Nota. Edición de un dispositivo.

Evidencia 9

Dispositivos con rol consulta

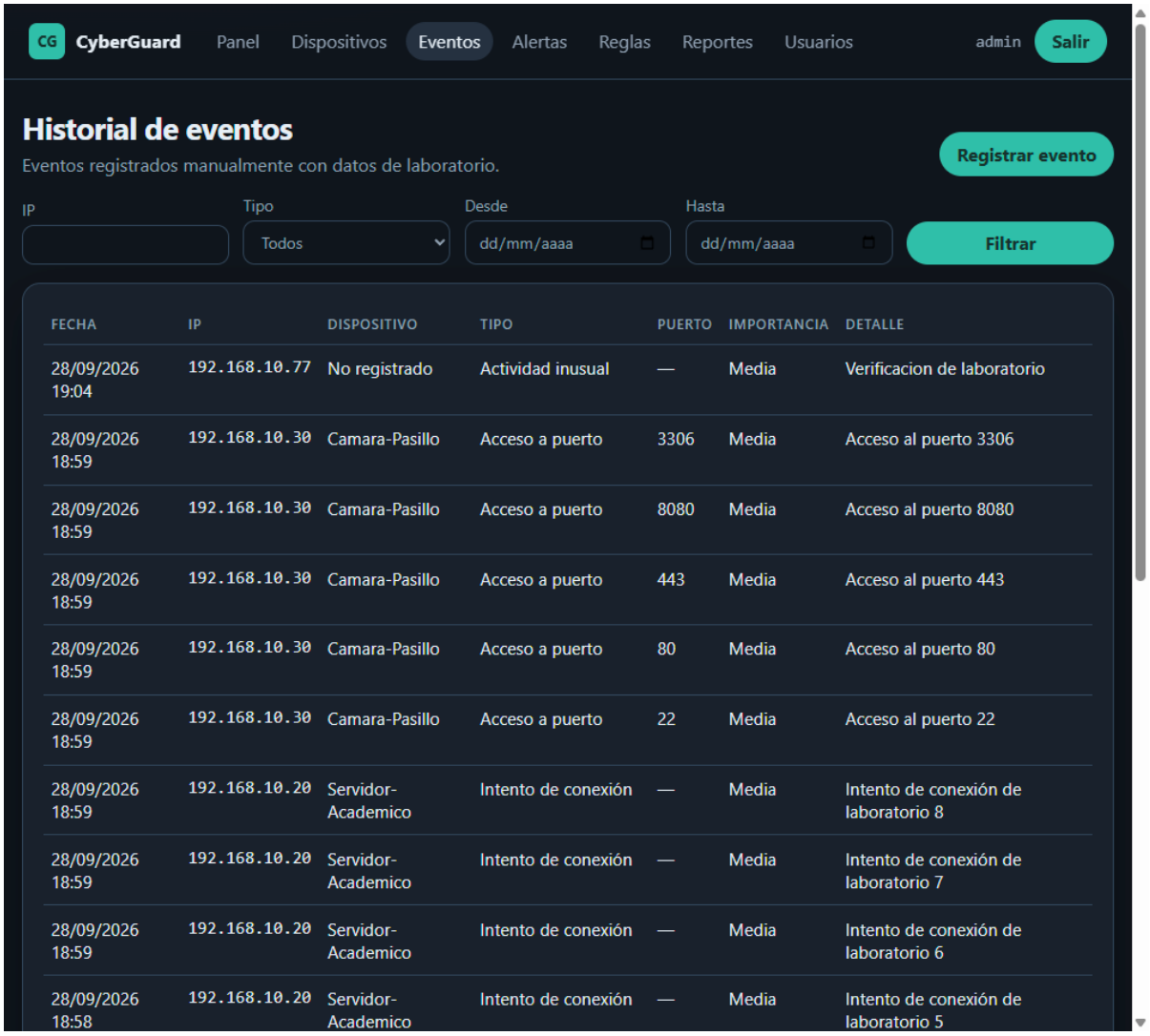


Nota. El rol consulta ve el inventario sin botones de creación ni edición.

14.4 Eventos

Evidencia 10

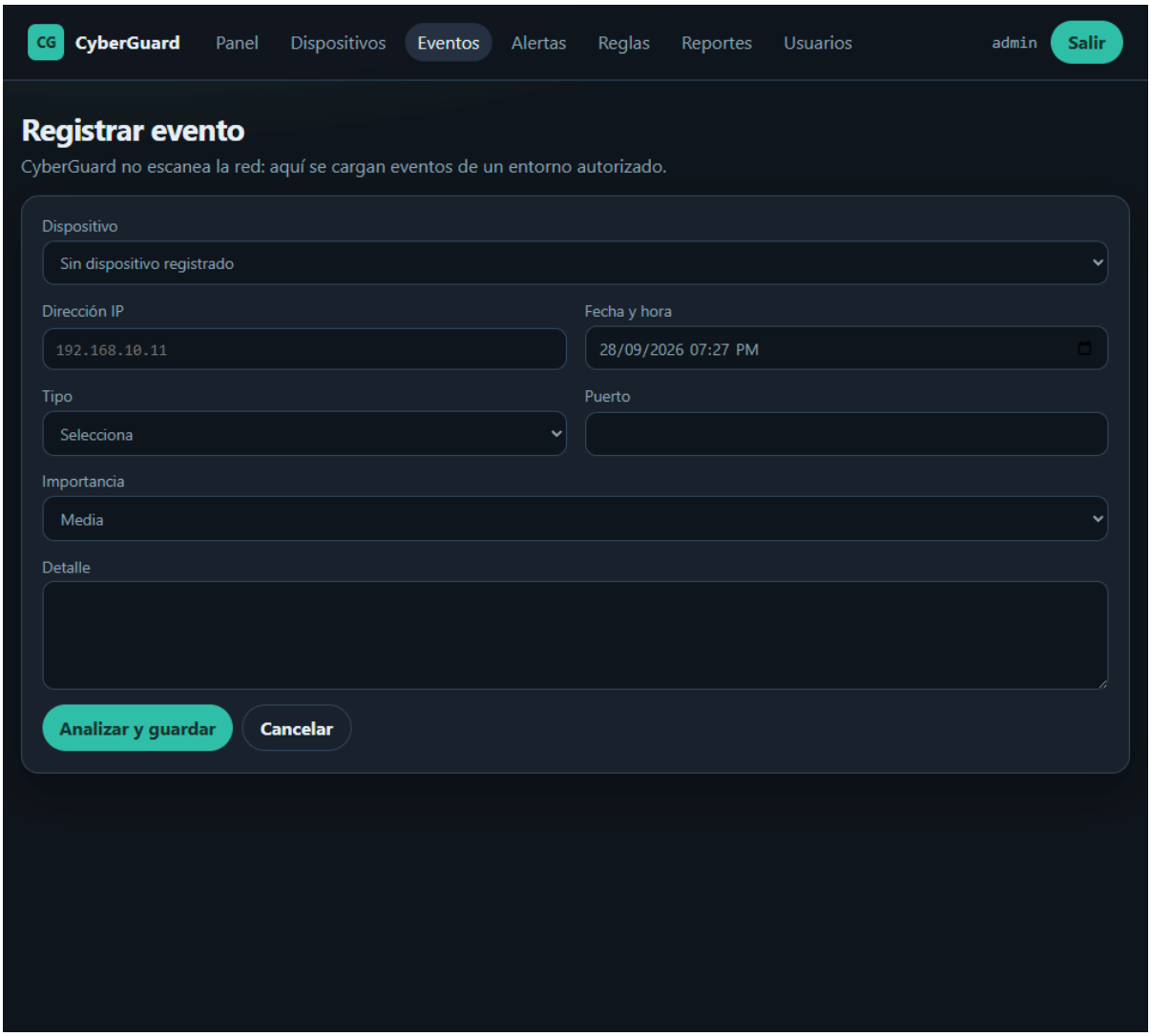
Historial de eventos



Nota. Historial de eventos con filtros.

Evidencia 11

Registrar evento

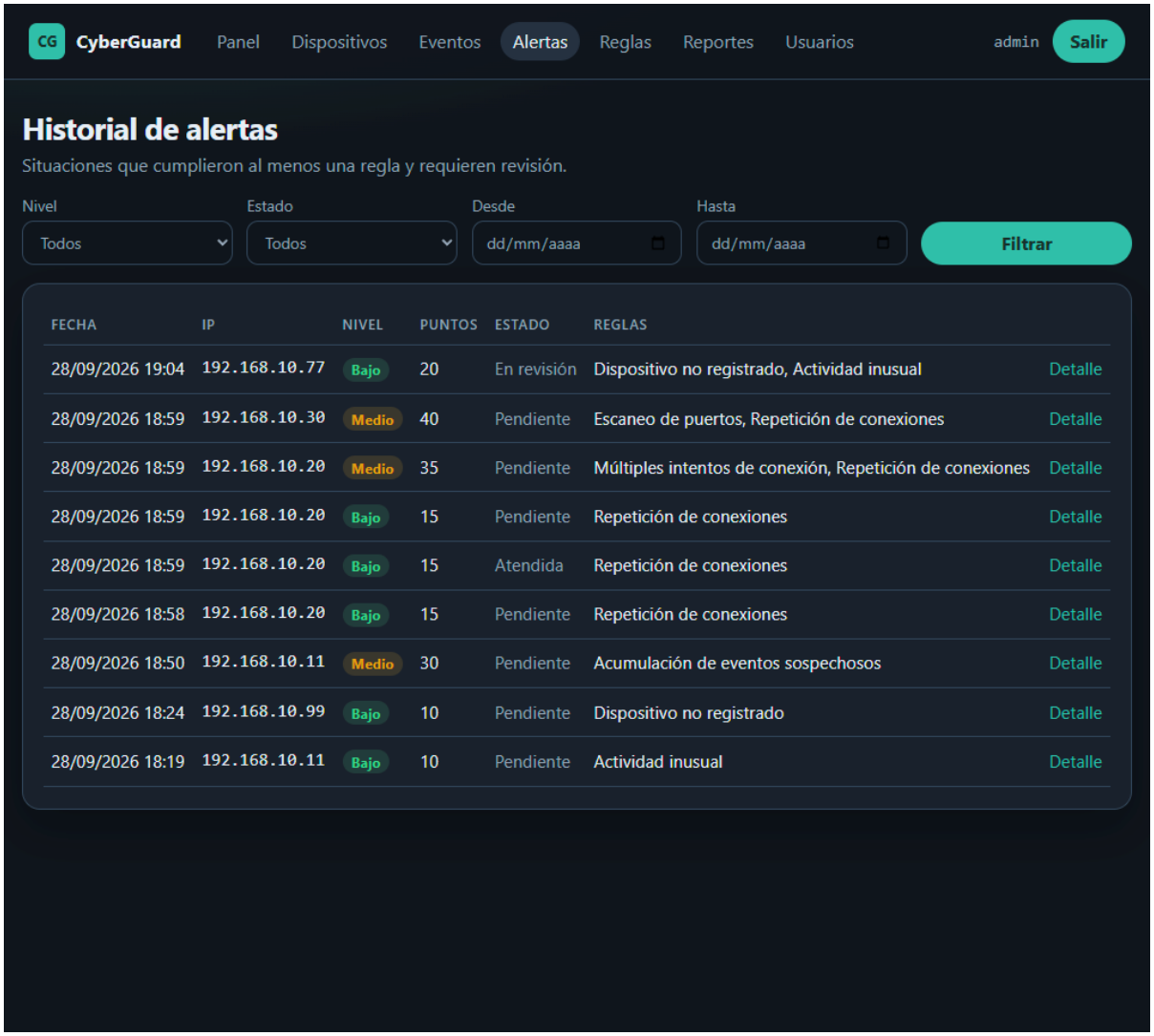


Nota. Formulario de registro de evento.

14.5 Alertas

Evidencia 12

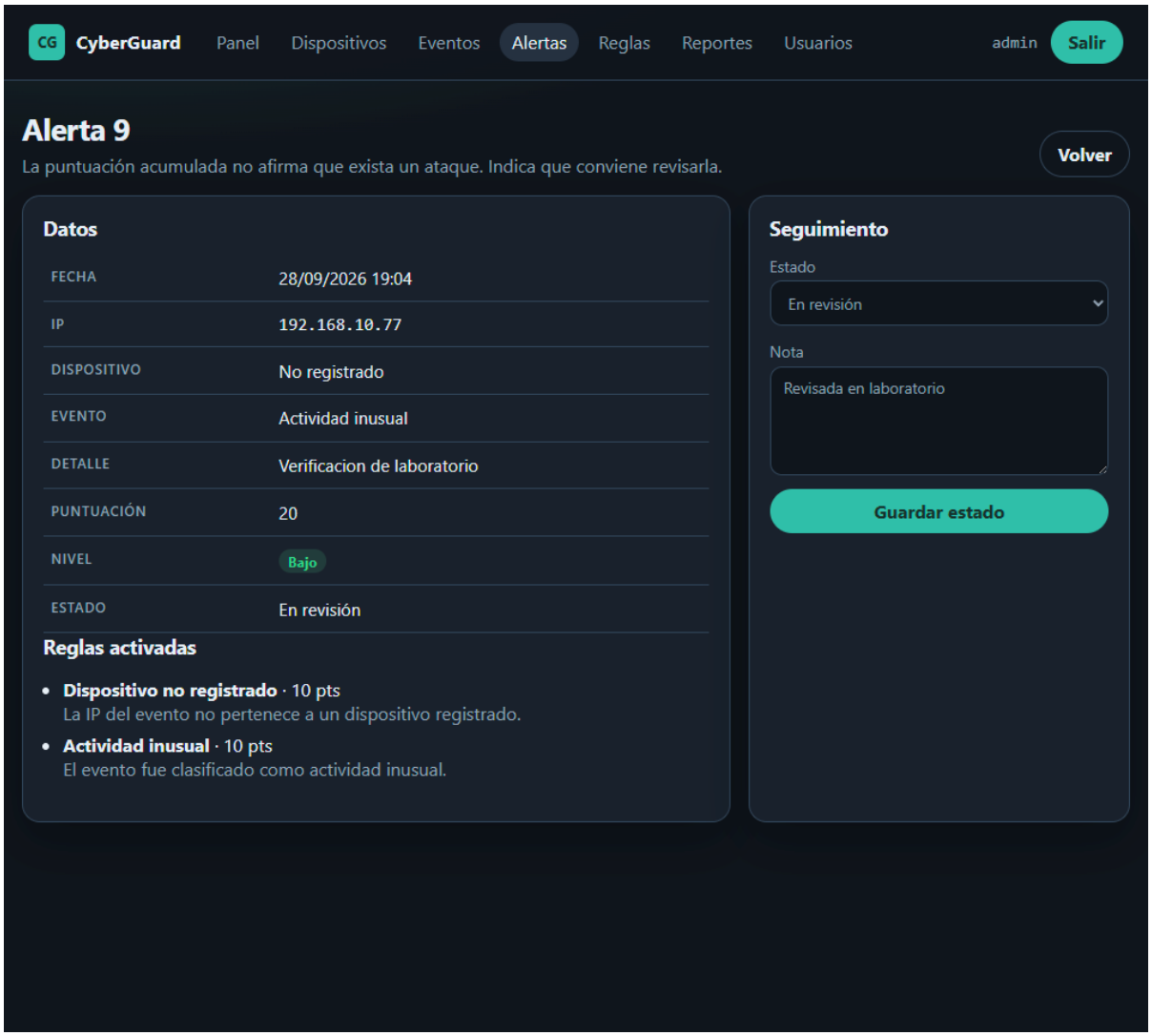
Lista de alertas



Nota. Lista de alertas con nivel y estado.

Evidencia 13

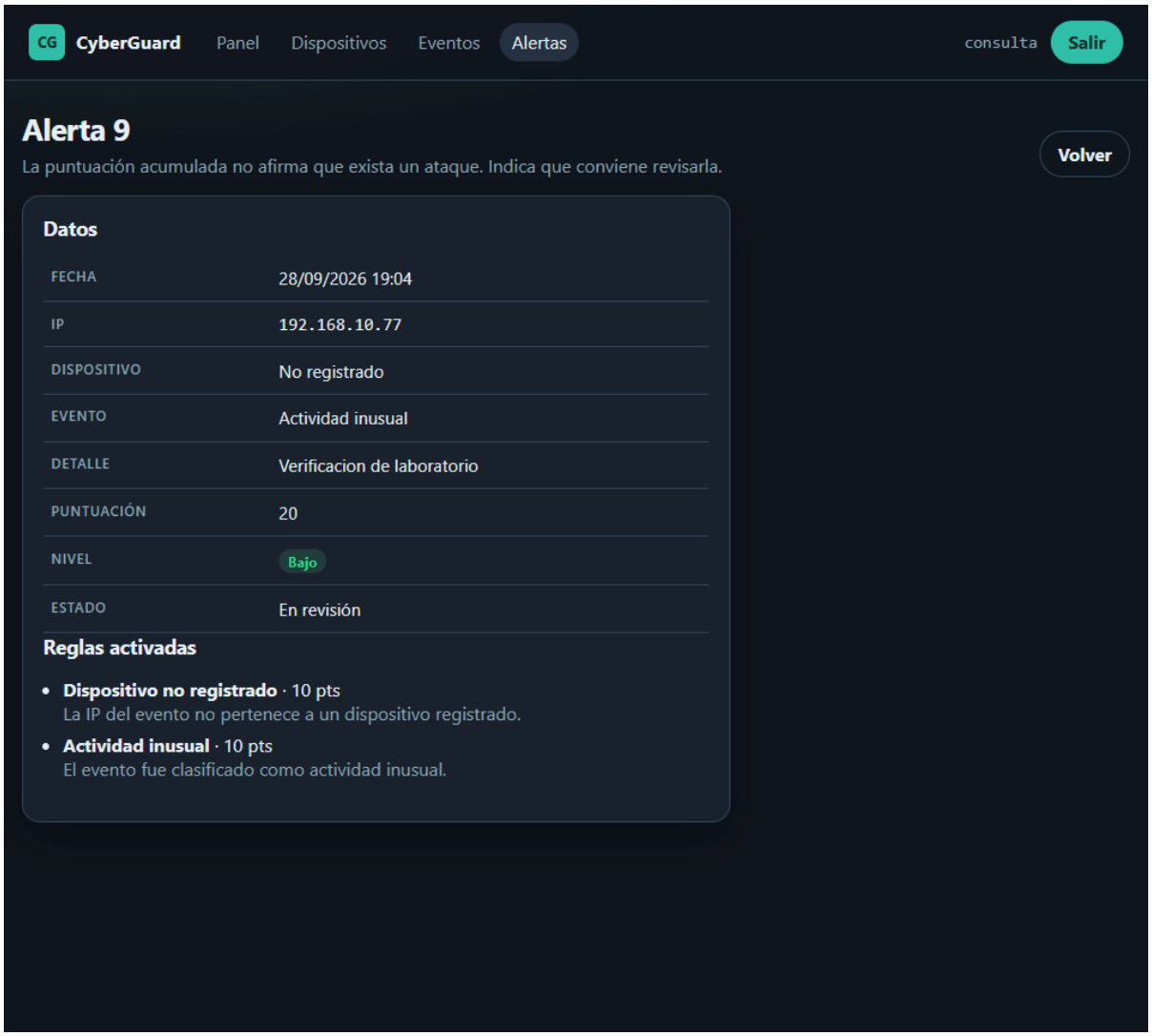
Detalle de alerta



Nota. Detalle de la alerta 9: IP 192.168.10.77 no registrada con actividad inusual; se cumplieron dos reglas (10 + 10 = 20 puntos, nivel bajo) y la alerta quedó «En revisión» con nota.

Evidencia 14

Detalle de alerta con rol consulta

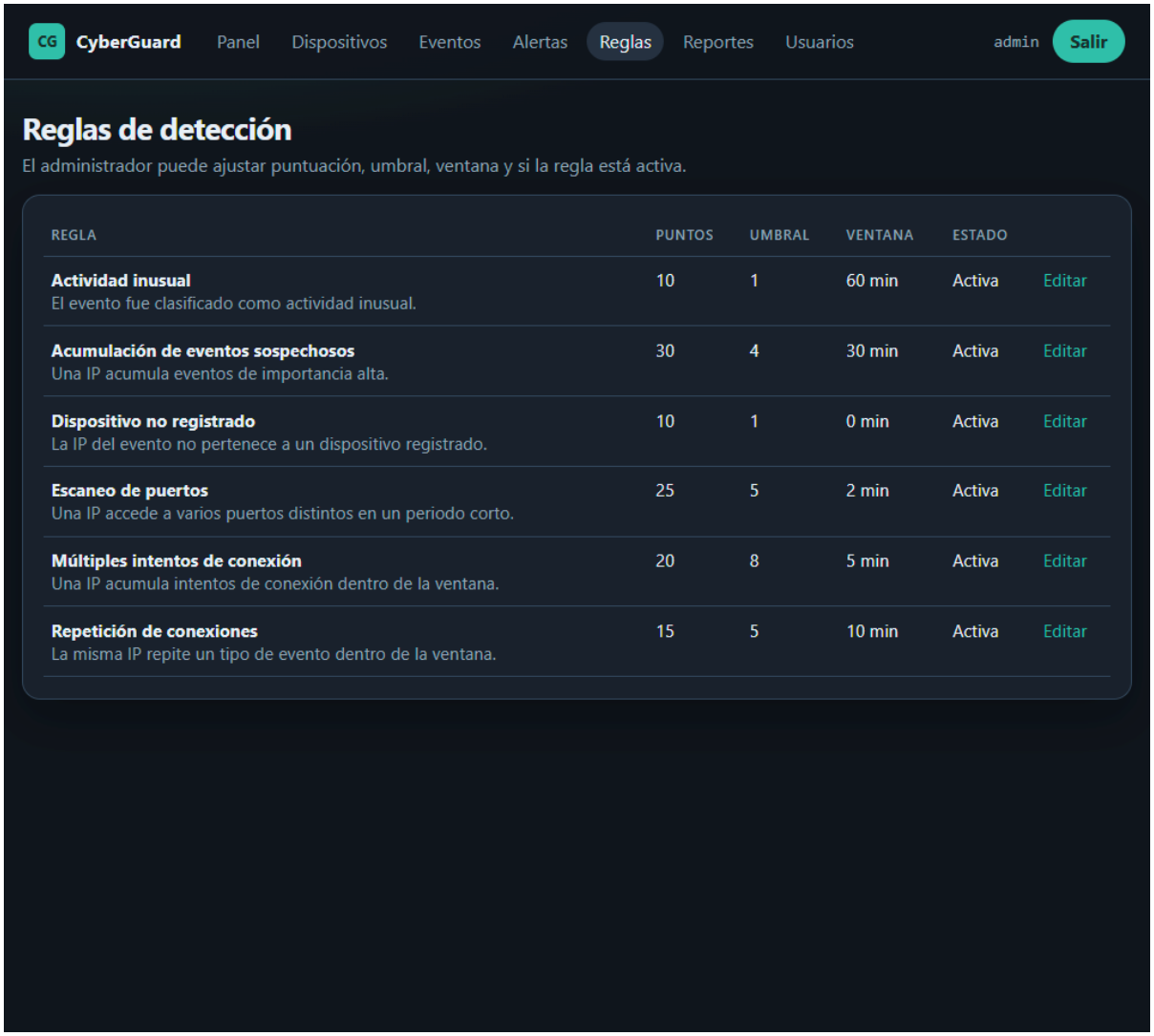


Nota. El rol consulta ve el detalle sin el formulario de seguimiento.

14.6 Reglas

Evidencia 15

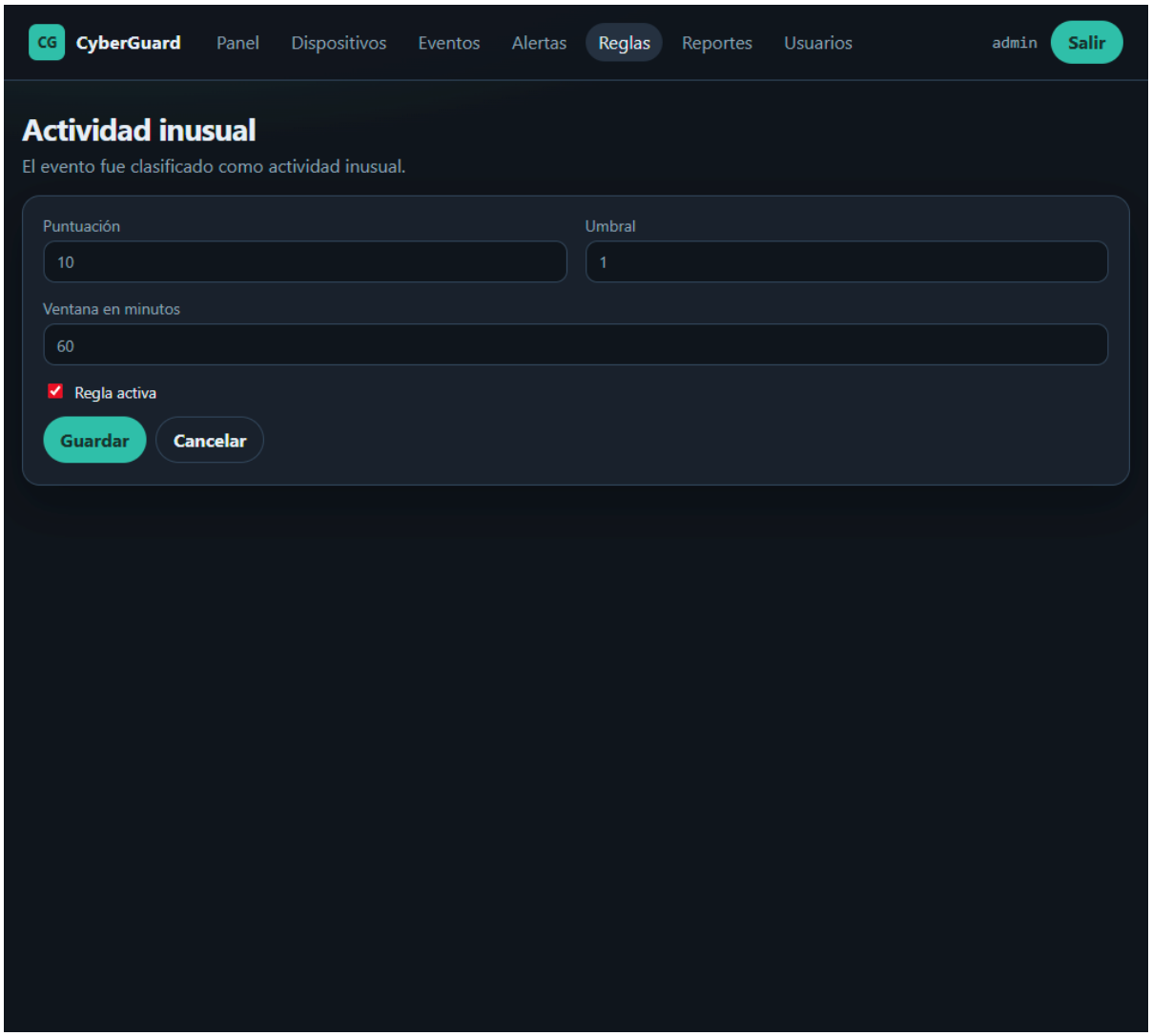
Catálogo de reglas



Nota. Catálogo de reglas con sus parámetros.

Evidencia 16

Editar regla

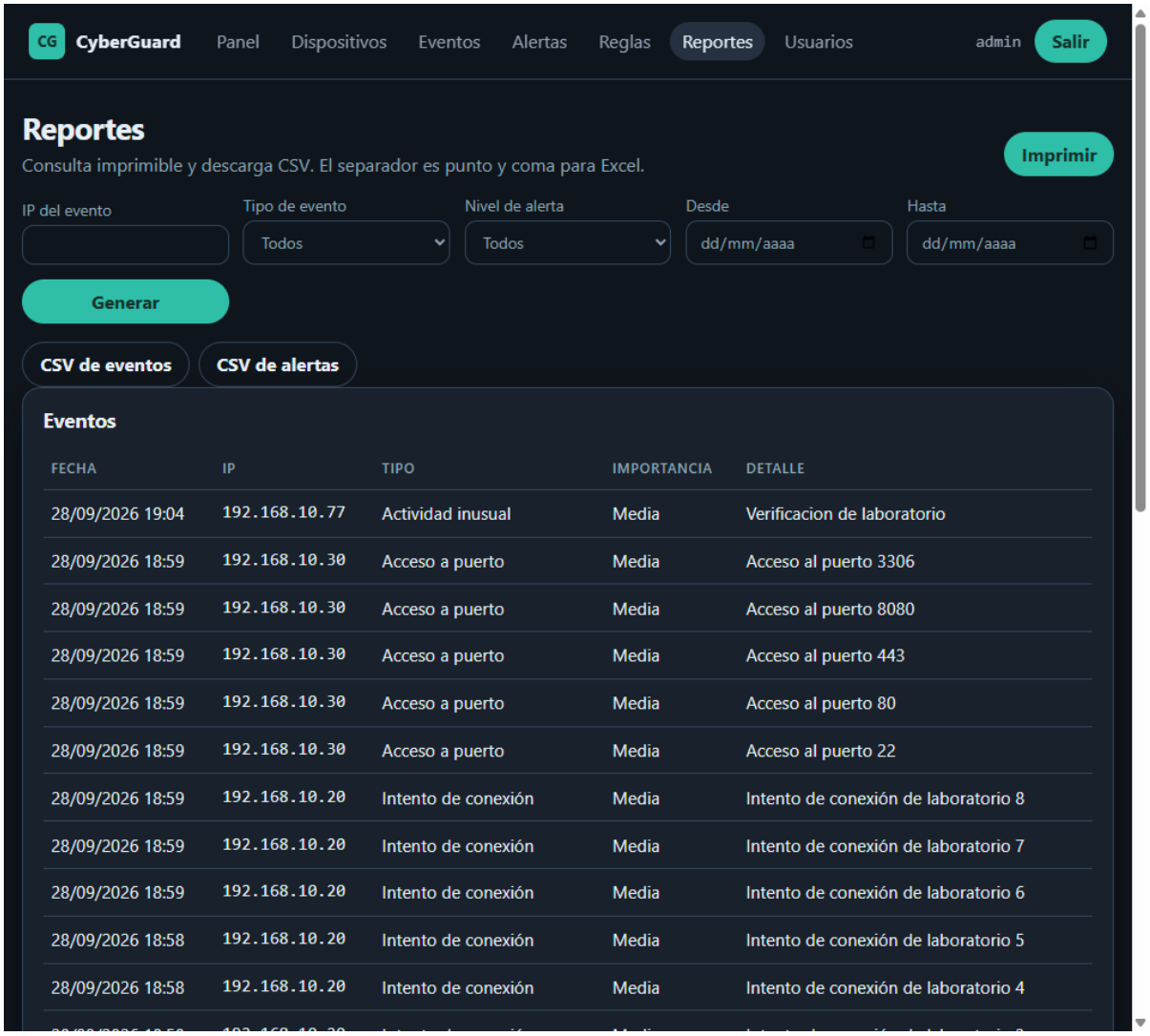


Nota. Edición de puntuación, umbral, ventana y activación.

14.7 Reportes

Evidencia 17

Reportes

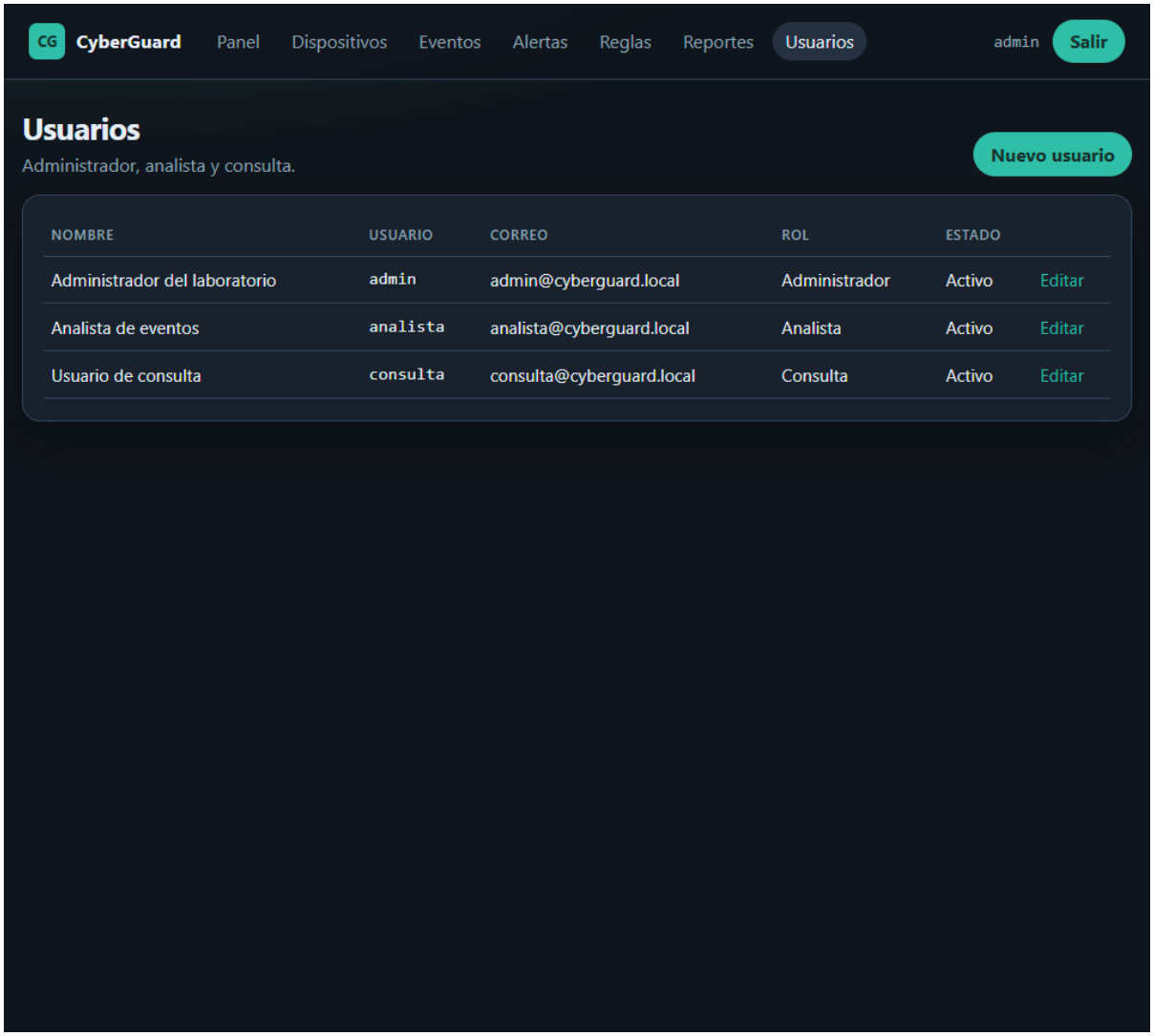


Nota. Consulta de reportes y descarga CSV.

14.8 Usuarios

Evidencia 18

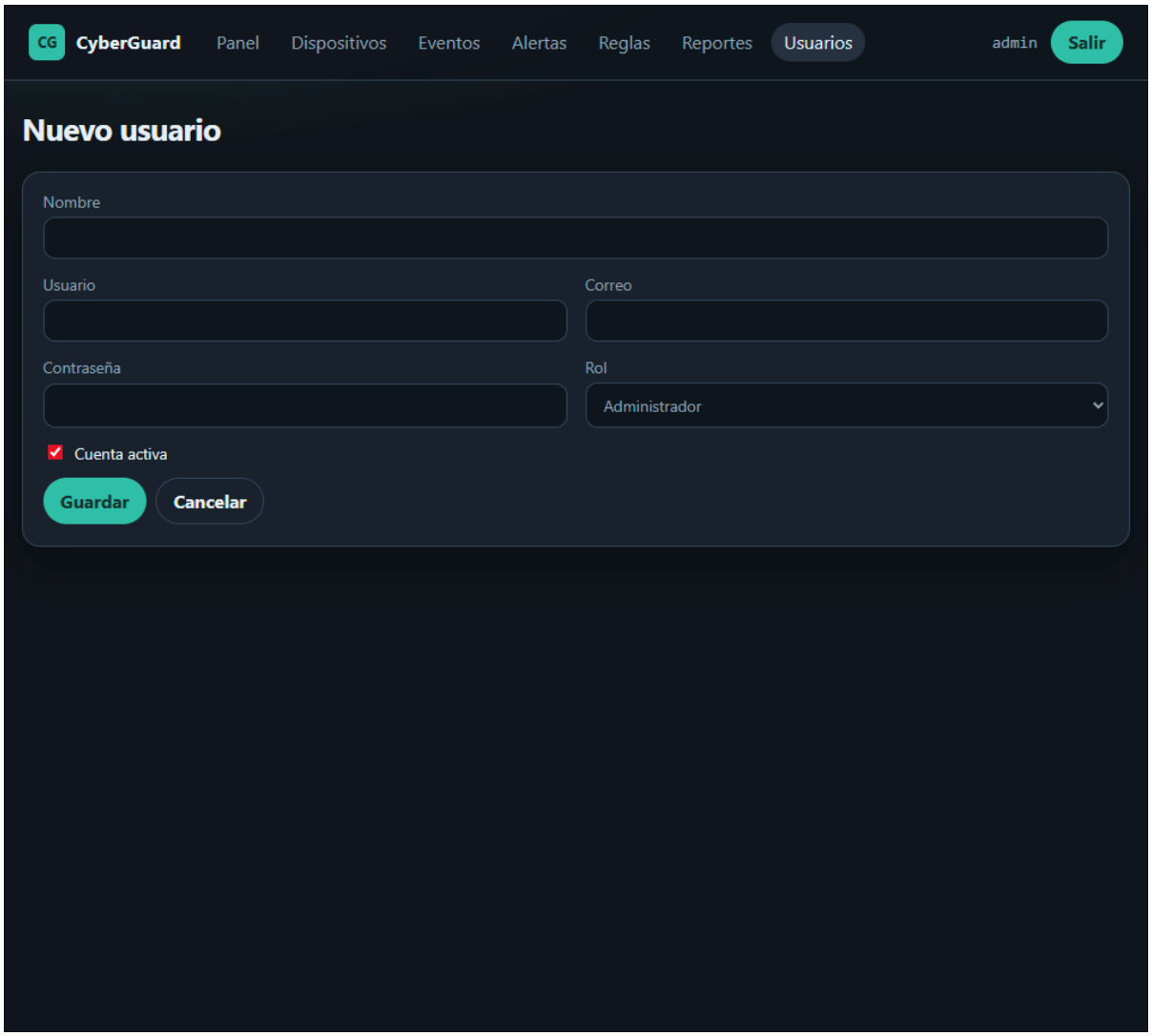
Lista de usuarios



Nota. Gestión de usuarios.

Evidencia 19

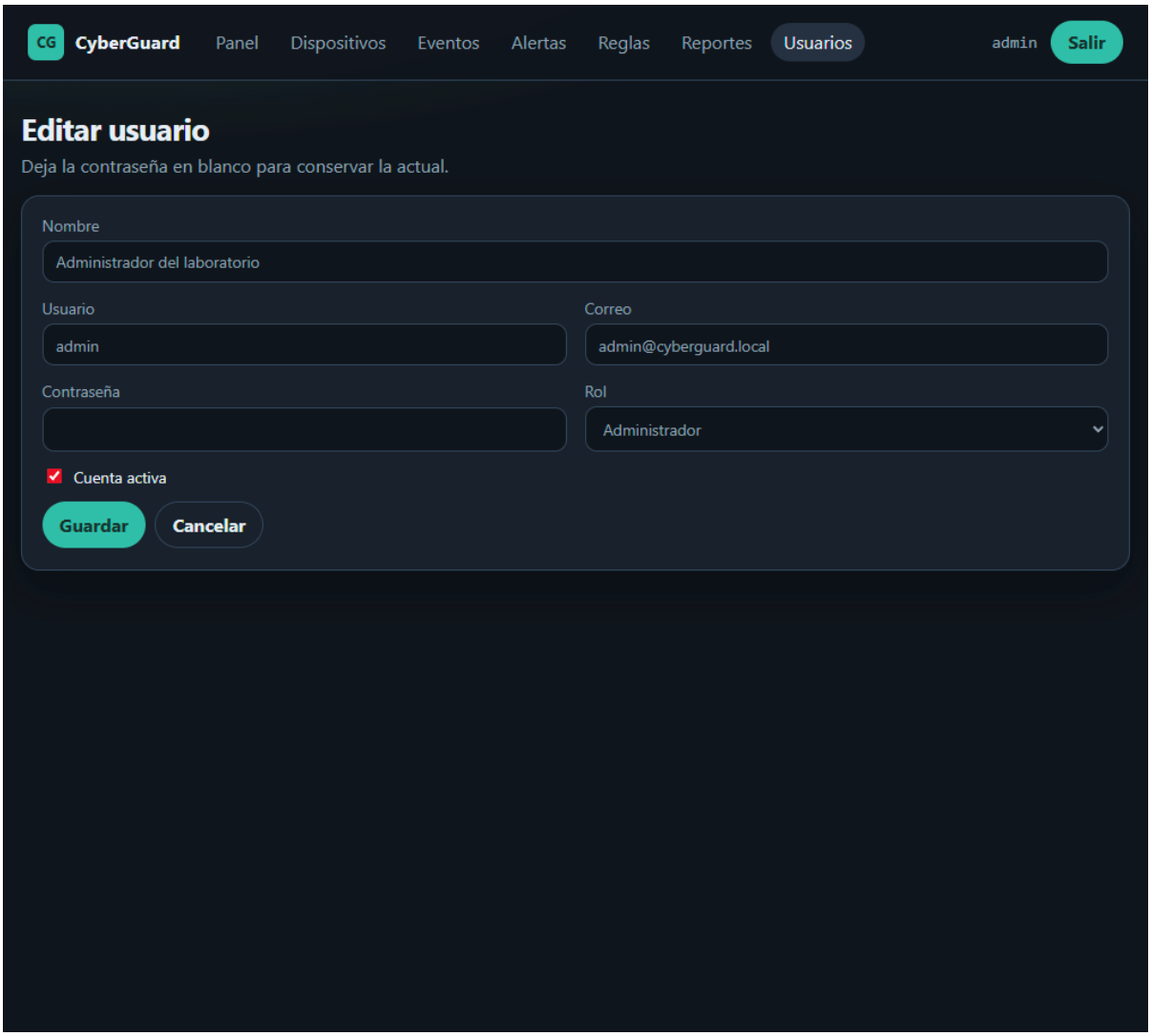
Nuevo usuario



Nota. Formulario de creación de usuario.

Evidencia 20

Editar usuario

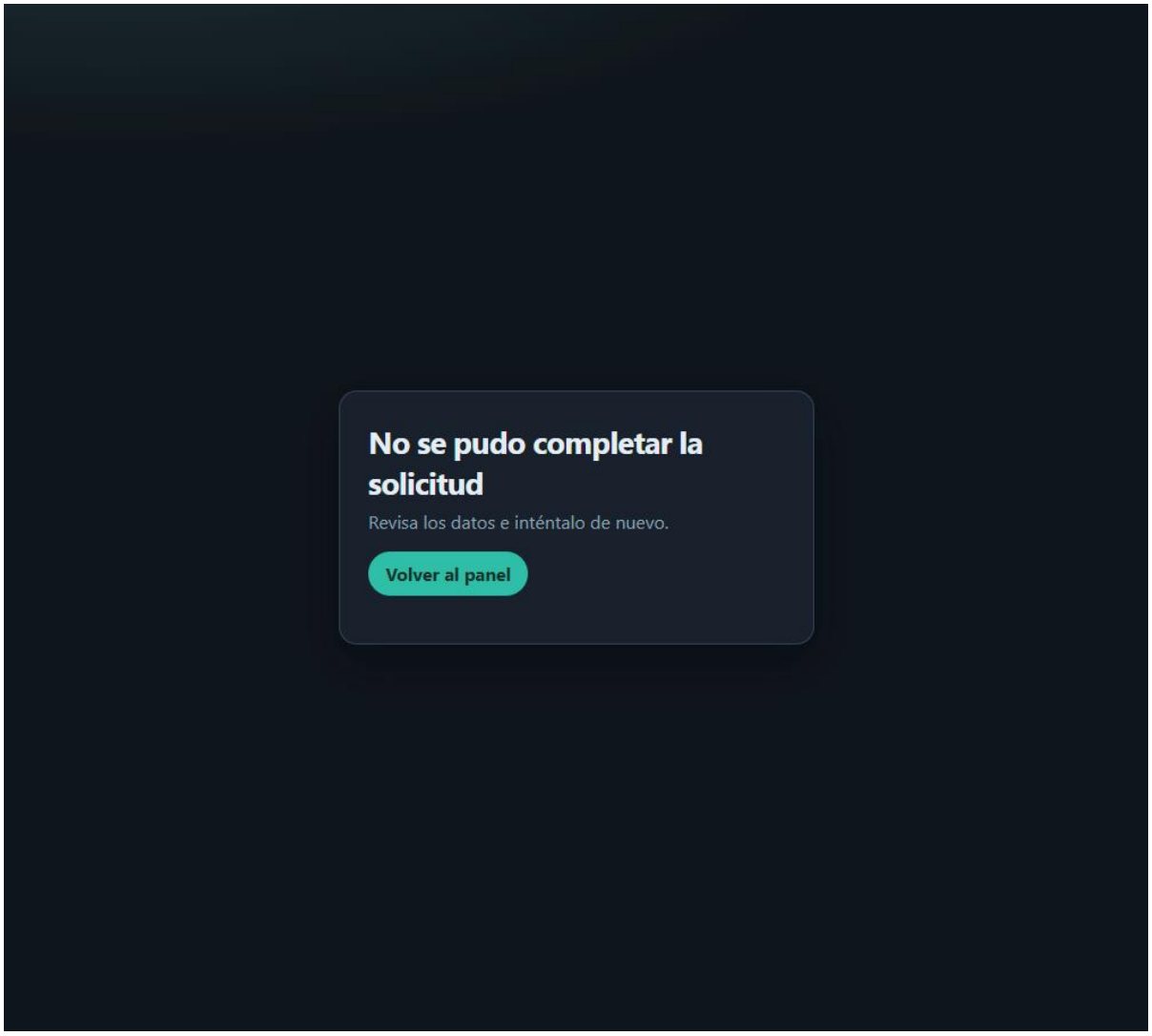


Nota. Edición de usuario.

14.9 Errores

Evidencia 21

Página no encontrada



Nota. Pantalla «Página no encontrada» para direcciones inexistentes.

15. Pruebas realizadas hasta el momento

15.1 Pruebas automáticas del backend

Se ejecutan con `mvnw.cmd test` sobre una base H2 en memoria (perfil `test`), sin necesidad de MySQL. Usan JUnit 5, Mockito, Spring Data JPA Test y MockMvc con Spring Security Test.

Tabla 18

Última ejecución: 5 de noviembre de 2026. Resultado: 62 pruebas, 0 fallos, 0 errores, 0 omitidas.

Clase de prueba	Tipo	Pruebas	Qué verifica	Resultado
`ClasificadorRiesgoTest`	Unitaria	2	Cortes de puntuación (bajo, medio, alto, crítico) y tope 0–100.	Aprobadas
`MotorAnalisisTest`	Unitaria con Mockito	4	Detecciones de ejemplo, suma de reglas y recorte en 100.	Aprobadas
`ReglasDeteccionTest`	Unitaria	13	Las seis reglas: caso que cumple, caso que no, y códigos no vacíos.	Aprobadas
`DispositivoTest`	Unitaria	2	`ultima_actividad` solo avanza; estado desconocido no registra.	Aprobadas
`EventoRepositoryTest`	Integración (JPA + H2)	2	Historial por IP, límites de ventana y exclusión de eventos posteriores.	Aprobadas
`EventoServiceTest`	Unitaria con Mockito	7	Última actividad, puerto obligatorio en acceso a puerto y creación de alerta.	Aprobadas
`UsuarioServiceTest`	Unitaria con Mockito	7	Alta con hash, edición sin pisar clave, unicidad y último administrador.	Aprobadas
`AlertaServiceTest`	Unitaria con Mockito	3	Crear con reglas, actualizar estado/nota y 404.	Aprobadas
`FechasTest`	Unitaria	3	Inicio del día y fin exclusivo de filtros.	Aprobadas
`EventoFormTest`	Bean Validation	2	Rechaza fecha futura; acepta pasada.	Aprobadas
`DispositivoFormTest`	Bean Validation	4	IP y MAC válidas o vacías.	Aprobadas
`UsuarioFormTest`	Bean Validation	3	Usuario y correo con formato correcto.	Aprobadas
`SeguridadAccesoTest`	Integración (MockMvc)	10	Login/token, 401/403/400/404 y recorrido de evento, alertas, panel y reportes.	Aprobadas

Salida resumida de Maven Surefire:

```
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in ClasificadorRiesgoTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- in MotorAnalisisTest
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0 -- in ReglasDeteccionTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in DispositivoTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in EventoRepositoryTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- in AlertaServiceTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 -- in EventoServiceTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- in FechasTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 -- in UsuarioServiceTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- in DispositivoFormTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in EventoFormTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- in UsuarioFormTest
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0 -- in SeguridadAccesoTest
Tests run: 62, Failures: 0, Errors: 0, Skipped: 0
```

15.1.1 Pruebas automáticas del cliente

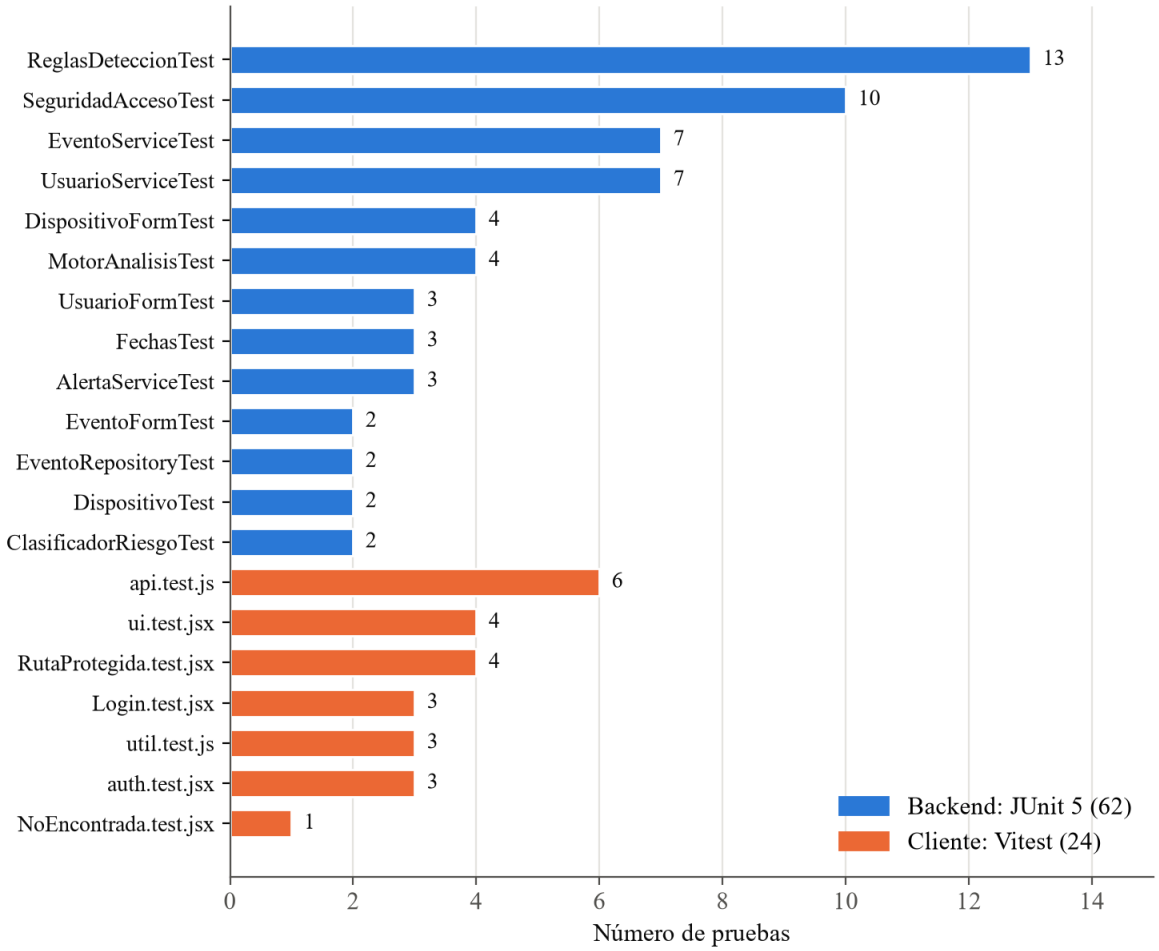
Se ejecutan con `npm test` en `frontend/` (Vitest + Testing Library + jsdom).

Última ejecución: 5 de noviembre de 2026. Resultado: 24 pruebas, 0 fallos.

La Figura 13 resume las 86 pruebas automáticas por clase o archivo. Las reglas de detección y el control de acceso de la API concentran la mayor cantidad, porque son las partes donde un error tiene más consecuencias: una regla mal evaluada genera alertas falsas o las omite, y un permiso mal configurado expone datos.

Figura 13

Pruebas automáticas por clase



Nota. Número de pruebas automáticas por clase del backend (JUnit 5) y por archivo del cliente (Vitest). Elaboración propia a partir de la ejecución del 5 de noviembre de 2026.

Tabla 19

Pruebas automáticas del cliente

Archivo	Pruebas	Qué verifica	Resultado
`api.test.js`	6	Sesión, query params, Bearer, 401/403 y fallo de red.	Aprobadas
`auth.test.jsx`	3	Permisos por rol, salir y entrar.	Aprobadas
`util.test.js`	3	`formatFecha` y `ahoraLocal`.	Aprobadas

Archivo	Pruebas	Qué verifica	Resultado
`RutaProtegida.test.jsx`	4	Redirección al login, acceso denegado y acceso por rol.	Aprobadas
`Login.test.jsx`	3	Sesión vigente, error de credenciales e ingreso correcto.	Aprobadas
`NoEncontrada.test.jsx`	1	Mensaje de página inexistente.	Aprobada
`ui.test.jsx`	4	`MensajeError`, `Estado` y `Campo`.	Aprobadas

15.2 Pruebas funcionales manuales

Se recorrió la aplicación con los tres usuarios de demostración. Las capturas de la sección 14 sirven de evidencia.

Tabla 20

Pruebas funcionales manuales

Caso	Pasos	Resultado esperado	Evidencia
PF01 Login correcto	Ingresar `admin` / `Admin123*`	Entra al panel	Evidencia 5
PF02 Login incorrecto	Ingresar una contraseña errada	Mensaje «Usuario o contraseña incorrectos», sigue en `/login`	Evidencia 2
PF03 Cerrar sesión	Pulsar «Salir»	Vuelve al login con aviso de sesión cerrada	Evidencia 3
PF04 Acceso por rol	Con `consulta`, abrir a mano una ruta de edición	Mensaje «Acceso denegado»	Evidencia 4
PF05 Interfaz por rol	Con `consulta`, abrir dispositivos y una alerta	Sin botones de edición ni formulario de seguimiento	Evidencias 9 y 14
PF06 Registro de dispositivo	Crear y editar un dispositivo	Aparece en el inventario	Evidencias 6 a 8
PF07 Registro de evento y alerta	Registrar actividad inusual desde 192.168.10.77	Se crea una alerta con dos reglas, 20 puntos, nivel bajo	Evidencia 13
PF08 Seguimiento	Cambiar la alerta a «En revisión» con nota	Estado y nota guardados	Evidencia 13
PF09 Ajuste de regla	Editar parámetros como administrador	Valores actualizados en el catálogo	Evidencias 15 y 16
PF10 Reportes	Filtrar y descargar CSV	Archivo con columnas separadas por punto y coma	Evidencia 17
PF11 Gestión de usuarios	Crear y editar un usuario	Aparece en la lista con su rol	Evidencias 18 a 20

15.3 Validaciones comprobadas

- IP con formato IPv4 válido; MAC con formato válido o vacía.
- Puerto entre 1 y 65535, obligatorio para «acceso a puerto».
- Fecha y hora del evento obligatorias y no futuras.
- Longitudes máximas en nombre, correo, detalle y nota.
- Usuario de 3 a 40 caracteres, solo letras, números, punto, guion y guion bajo.
- Correo válido y único; IP de dispositivo única.
- Puntuación de regla entre 1 y 100, umbral mínimo 1, ventana entre 0 y 1 440 minutos.

16. Problemas encontrados y cómo fueron solucionados

Tabla 21

Problemas encontrados y cómo fueron solucionados

#	Problema o riesgo identificado	Causa	Solución aplicada
1	Hibernate fallaba al arrancar con la base de XAMPP.	XAMPP trae MariaDB y Hibernate 6 no puede leer su metadata JDBC (falla con la columna `RESERVED`).	Se fijó el dialecto a mano (`MySQLDialect`) y se desactivó la lectura de metadata (`hibernate.boot.allow_jdbc_metadata_access=false`).
2	Había que crear la base a mano antes de ejecutar.	MySQL no crea bases de datos por sí solo.	La URL JDBC incluye `createDatabaseIfNotExist=true`; Hibernate crea las tablas con `ddl-auto=update`.
3	Los reportes CSV no se podían descargar con un enlace.	Con JWT, un enlace `<a href>` no envía la cabecera `Authorization` y la API responde 401.	El cliente pide el archivo con `fetch` y el token, y lo guarda desde el navegador (`descargar` en `api.js`). La API expone `Content-Disposition` en CORS.

#	Problema o riesgo identificado	Causa	Solución aplicada
4	Error de carga perezosa al ver el detalle de una alerta.	Con `open-in-view` desactivado, las reglas de la alerta (relación `LAZY`) no se pueden cargar al convertir la respuesta en JSON.	`AlertaService.obtener` usa una consulta con las relaciones cargadas (`findConDetalle`) e inicializa las reglas dentro de la transacción.
5	Riesgo de sobrescribir la contraseña al editar un usuario.	Llenar la entidad directamente desde el formulario pisaría el hash guardado.	Formularios separados (`UsuarioForm`); la contraseña solo se cambia si el campo viene con texto.
6	Un administrador podía quedarse sin acceso.	Nada impedía desactivar al último administrador o a uno mismo.	`UsuarioService.validarAdministradorRestante` exige que quede al menos un administrador activo y bloquea la autodesactivación.
7	Los datos de demostración se duplicaban en cada reinicio.	La carga inicial se ejecutaba siempre.	`DataInitializer` solo escribe si la tabla correspondiente está vacía.
8	Con datos de ejemplo insertados directamente en la base, el panel no mostraría alertas ni el motor en acción.	Una inserción directa no pasa por el análisis.	Los eventos de ejemplo se registran con `EventoService`, así que pasan por el motor y generan alertas reales.
9	Los CSV mostraban mal las tildes y las columnas en Excel.	Excel en español espera punto y coma y no detecta UTF-8 sin marca.	Separador `;`, marca BOM al inicio y escape de comillas y saltos de línea.
10	Eventos de acceso a puerto sin puerto hacían inútil la regla de escaneo.	El puerto es opcional para los demás tipos.	`EventoService.registrar` exige puerto cuando el tipo es «acceso a puerto».
11	La aplicación no arrancaba en algunos equipos (`UnsupportedClassVersionError`).	El JDK instalado era anterior a 21.	Se documentó el requisito de JDK 21 o superior y cómo configurar `JAVA_HOME` en `Docs/instalacion.md`.
12	Ejecutar las pruebas no debía depender de tener MySQL encendido.	La configuración principal apunta a MySQL local.	Perfil `test` con H2 en memoria en modo MySQL y `ddl-auto=create-drop`.
13	Posible desfase de las fechas de eventos respecto a la hora local.	La zona horaria del servidor de base de datos puede diferir de la de la JVM.	Zona `America/Bogota` en la URL JDBC y en `hibernate.jdbc.time_zone`.

#	Problema o riesgo identificado	Causa	Solución aplicada
14	El navegador bloqueaba las llamadas del cliente a la API.	Cliente (5173) y API (8080) tienen orígenes distintos, y el navegador aplica CORS.	En desarrollo, Vite reenvía `/api` a la API; para una publicación real, la API acepta los orígenes de `CYBERGUARD_CORS_ORIGENES`.
15	Ante un 401 o 403 el cliente no tenía qué mensaje mostrar.	Spring Security responde esos códigos con el cuerpo vacío.	`api.js` usa un mensaje por código de estado cuando no llega cuerpo y, ante un 401, cierra la sesión.
16	Un evento registrado con fecha pasada podía disparar reglas de más.	La consulta del historial solo tenía límite inferior (`fechaHora >= desde`) y contaba también los eventos posteriores de la misma IP.	La ventana termina en la fecha del evento (`fechaHora <= hasta`) y las fechas futuras se rechazan en la API y en el cliente. Lo cubren `EventoRepositoryTest` y `EventoFormTest`.
17	Credenciales y clave JWT escritas en `application.properties`.	Cada equipo tiene su propia contraseña de MySQL y la clave no debe ir al repositorio.	Los valores salen de `backend/.env` (ignorado por Git) con spring-dotenv; `backend/.env.example` sirve de plantilla.
18	Un evento con fecha pasada hacía retroceder la última actividad del dispositivo.	`ultima_actividad` se reescribía con cualquier fecha del evento ligado.	`Dispositivo.registrarActividad` solo avanza la fecha; lo cubre `EventoServiceTest` y `DispositivoTest` (ver D5 en `NORMALIZACION.md`).

Observación del análisis con los datos de demostración. Con los valores iniciales, una sola regla nunca supera 30 puntos, así que el panel muestra 0 alertas altas y 0 críticas (Evidencia 5). Para llegar a alto o crítico deben cumplirse varias reglas en el mismo evento. Es coherente con el diseño (el riesgo alto nace de la combinación de señales), pero se revisará la calibración de puntos en la tercera entrega.

17. Funcionalidades pendientes para la tercera entrega

Tabla 22

Funcionalidades pendientes para la tercera entrega

Prioridad	Funcionalidad	Descripción
Alta	Cobertura y ampliación de pruebas	Ya hay 62 pruebas del backend (reglas, servicios, formularios, dominio y `SeguridadAccesoTest`) y 24 del cliente. Falta medir cobertura con JaCoCo y ampliar pantallas de negocio del cliente (dispositivos, eventos, alertas, reportes).
Alta	Auditoría de acciones	Registrar qué usuario registró cada evento y quién cambió el estado de cada alerta, con fecha (relación entre `usuarios`, `eventos` y `alertas`).
Alta	Transiciones de estado controladas	Aplicar en `AlertaService` el flujo de la Figura 8 en lugar de permitir cualquier cambio.
Media	Carga masiva de eventos	Importar eventos desde un archivo CSV generado en el laboratorio (por ejemplo, exportaciones de Wireshark o de registros de un equipo autorizado) y analizarlos uno por uno.
Media	Calibración de reglas	Probar con conjuntos de datos del laboratorio y ajustar puntos, umbrales y ventanas para que los niveles alto y crítico sean alcanzables en escenarios realistas.
Media	Gráficas en el panel	Alertas por nivel, eventos por tipo y actividad por hora.
Media	Paginación e índices	Paginar las listas de eventos y alertas y crear un índice sobre `eventos(direccion_ip, fecha_hora)`, que es la consulta del historial que usa el motor.
Baja	Reportes en PDF	Además del CSV, un reporte imprimible.
Baja	Política de contraseñas y bloqueo por intentos	Exigir complejidad mínima y bloquear temporalmente tras varios intentos fallidos.
Media	Ampliar pruebas del cliente web	Base con Vitest y Testing Library ya creada (`npm test`). Falta cubrir formularios y listados de dispositivos, eventos, alertas y reportes.
Baja	Revocación de sesiones	Comprobar en cada petición que la cuenta siga activa y con el mismo rol, o acortar la vida del token, para que desactivar una cuenta tenga efecto inmediato.
Baja	Despliegue de demostración	Empaquetar el `.jar` y `frontend/dist`, y documentar su ejecución en un equipo del laboratorio para la sustentación.
—	Documento final	Resultados de las pruebas ampliadas, manual de usuario y conclusiones finales.

18. Conclusiones parciales

1. CyberGuard pasó de propuesta a aplicación funcional. Los 13 requerimientos funcionales y los 8 no funcionales tienen implementación en el código, y los módulos planeados en la primera entrega (usuarios, dispositivos, eventos, motor de análisis, reglas, alertas, panel y reportes) están integrados.
2. El motor de análisis es la parte central del proyecto y cumple su propósito: cada evento se evalúa en el momento del registro contra reglas configurables, y el resultado queda como una alerta con puntuación, nivel y reglas cumplidas. El patrón estrategia permitió separar la condición (en Java) de los parámetros (en la base), de modo que el administrador ajusta el comportamiento sin tocar código y agregar una regla nueva no obliga a modificar el motor.
3. La arquitectura por capas facilitó el trabajo en paralelo y las pruebas, y permitió separar el cliente de la API sin tocar el motor ni los servicios: el cambio se concentró en la capa de presentación y en la seguridad. El motor se probó de forma aislada con Mockito, y la API con MockMvc sin depender de MySQL ni de un navegador.
4. La seguridad se aplicó en el propio software y no solo como tema del proyecto: roles por endpoint, tokens JWT firmados, contraseñas con BCrypt, validación de entradas, configuración sensible fuera del código y protecciones para no perder el acceso administrativo.
5. Varios problemas técnicos (MariaDB con Hibernate 6, carga perezosa sin `open-in-view`, CORS entre cliente y API, descarga de archivos con token, formato CSV para Excel, la ventana del historial) se resolvieron y quedaron documentados, lo que reduce el tiempo de instalación en otros equipos.

6. Las 62 pruebas del backend y las 24 del cliente cubren clasificación de riesgo, el motor y cada regla, servicios de evento/usuario/alerta, validaciones de formularios, autenticación y permisos de la API, más api/auth/rutas del cliente. Sigue pendiente medir cobertura con JaCoCo y ampliar pantallas de negocio; junto con la auditoría de acciones y la calibración de las reglas, eso prioriza la tercera entrega.
7. El proyecto se mantiene dentro de su alcance académico: registra y analiza eventos de una red autorizada, no escanea ni bloquea, y presenta sus alertas como señales para revisión humana y no como dictámenes de ataque.

19. Referencias

Chandola, V., Banerjee, A. y Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*(3), Artículo 15. https://doi.org/10.1145/1541880.1541882

Chen, P. P.-S. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems, 1*(1), 9–36. https://doi.org/10.1145/320434.320440

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

JUnit Team. (s. f.). *JUnit 5 user guide*. https://junit.org/junit5/docs/current/user-guide/

Kent, K. y Souppaya, M. (2006). *Guide to computer security log management* (NIST Special Publication 800-92). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-92

Liao, H.-J., Lin, C.-H. R., Lin, Y.-C. y Tung, K.-Y. (2013). Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications, 36*(1), 16–24. https://doi.org/10.1016/j.jnca.2012.09.004

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Mermaid. (s. f.). *Mermaid: Diagramming and charting tool*. https://mermaid.js.org/

Meta Open Source. (s. f.). *React documentation*. https://react.dev/

Mockito. (s. f.). *Mockito framework*. https://site.mockito.org/

National Institute of Standards and Technology. (2012). *Computer security incident handling guide* (NIST Special Publication 800-61 Rev. 2). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-61r2

National Institute of Standards and Technology. (2024). *The NIST Cybersecurity Framework (CSF) 2.0* (NIST CSWP 29). U.S. Department of Commerce. https://doi.org/10.6028/NIST.CSWP.29

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Oracle. (s. f.). *Java documentation*. https://docs.oracle.com/en/java/

Oracle Corporation. (s. f.). *MySQL 8.4 reference manual*. https://dev.mysql.com/doc/refman/8.4/en/

OWASP Foundation. (2021a). *OWASP Top 10: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/

OWASP Foundation. (2021b). *OWASP Application Security Verification Standard 4.0.3*. https://owasp.org/www-project-application-security-verification-standard/

Provos, N. y Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the 1999 USENIX Annual Technical Conference* (pp. 81–91). USENIX Association.

Scarfone, K. y Mell, P. (2007). *Guide to intrusion detection and prevention systems (IDPS)* (NIST Special Publication 800-94). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-94

Schwaber, K. y Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

Spring. (s. f.-a). *Spring Security reference*. https://docs.spring.io/spring-security/reference/

Spring. (s. f.-b). *Spring Data JPA reference documentation*. https://docs.spring.io/spring-data/jpa/reference/

Vite. (s. f.). *Vite guide*. https://vite.dev/guide/

Vitest. (s. f.). *Vitest documentation*. https://vitest.dev/

VMware. (s. f.). *Spring Boot documentation*. https://docs.spring.io/spring-boot/index.html

Anexo A. Índice de figuras

Tabla 23

Anexo A. Índice de figuras

Figura	Título	Archivo
1	Permisos por rol	`Docs/img/10-permisos-roles.png`
2	Diagrama de casos de uso	`Docs/img/01-casos-de-uso.png`
3	Diagrama de clases del dominio	`Docs/img/03-clases-dominio.png`
4	Diagrama de clases del motor de análisis	`Docs/img/04-clases-motor.png`
5	Modelo entidad-relación	`Docs/img/05-entidad-relacion.png`
6	Diagrama de secuencia del registro de un evento	`Docs/img/06-secuencia-registro-evento.png`
7	Diagrama de actividades del análisis	`Docs/img/07-actividad-analisis.png`
8	Diagrama de estados de la alerta	`Docs/img/08-estados-alerta.png`
9	Diagrama de despliegue	`Docs/img/09-despliegue.png`
10	Puntuación por regla y cortes de riesgo	`Docs/img/12-puntuacion-reglas.png`
11	Arquitectura por capas	`Docs/img/02-arquitectura-capas.png`
12	Líneas de código por componente	`Docs/img/13-lineas-codigo.png`
13	Pruebas automáticas por clase	`Docs/img/11-pruebas-por-clase.png`

Las fuentes editables de los diagramas 1 a 9 y 11 están en `Docs/img/fuentes/` (formato Mermaid `.mmd`). Las gráficas 10, 12 y 13 se generaron con Python y Matplotlib a partir de los datos del repositorio. Para regenerar una imagen:

```
npx -y @mermaid-js/mermaid-cli -i Docs\img\fuentes\01-casos-de-uso.mmd -o Docs\img\01-casos-de-uso.png -b white -s 2
```